



Faculty of engineering, Cairo University
Electronics and Electrical Communication Department
ELC4020 – Digital Communication



Digital Communication

OFDM Project

ELC 4020 | Fall 2025

Submitted to:

Dr. Mohamed Khairy

Eng. Mohammed Khaled

<i>Name</i>	<i>Id</i>
روى عاطف محمد عبدالمجيد	9220291
ساره عبدالعاطي ابراهيم عبدالعاطي	9220349
سما سعيد محمد محمد	9220371

Table of Content

Problem 1: Execution time of DFT and FFT	4
1. Results	4
Problem 2: Bit-error rate performance for BPSK and 16-QAM over Rayleigh flat fading channel	5
1. BPSK	5
1.1. BPSK uncoded vs coded (same energy per information bit) Results	5
1.2. BPSK uncoded vs coded (same energy per transmission bit) Results	6
1.3. BPSK with Same Transmission Energy Vs. BPSK with Same Information Energy	7
2. QPSK	8
2.1. QPSK uncoded vs coded (same energy per information bit) Results	8
2.2. QPSK uncoded vs coded (same energy per transmission bit) Results	9
2.3. QPSK with Same Transmission Energy Vs. QPSK with Same Information Energy	10
3. 16-QAM	11
3.1. 16-QAM uncoded vs coded (same energy per information bit) Results	11
3.2. 16-QAM uncoded vs coded (same energy per transmission bit) Results	12
3.3. 16-QAM with Same Transmission Energy Vs. 16-QAM with Same Information Energy	13
4. Uncoded Modulation Comparison (Same Transmission Energy) Results	14
5. Uncoded Modulation Comparison (Same information Energy) Results	15
6. Coded Modulation Comparison (Same Transmission Energy) Results	16
7. Coded Modulation Comparison (Same information Energy) Results	17
8. Coded Modulation Comparison (Same information Energy) Results	18
Problem 3: OFDM System Simulation	20
1. QPSK With Rayleigh Flat Fading Channel	20
2. QPSK With Frequency Selective Channel	21
3. 16-QAM With Rayleigh Flat Fading Channel	22
4. 16-QAM With Frequency Selective Channel	23
Appendix : Full Matlab code	24

Table of Figures

Figure 1 : FFT & DFT execution times	4
Figure 2 : BPSK same energy per info bit result	5
Figure 3 : BPSK same energy per transmission bit result	6
Figure 4 : BPSK Info vs. Transmission	7
Figure 5 : QPSK same energy per info bit result	8
Figure 6 : QPSK same energy per transmission bit result	9
Figure 7 : QPSK Info vs. Transmission	10
Figure 8 : 16-QAM same energy per info bit result	11
Figure 9 : 16-QAM same energy per transmission bit	12
Figure 10 : 16-QAM Info vs. Transmission	13
Figure 11 : uncoded modulation result (same transmission energy)	14
Figure 12 : uncoded modulation result (same information energy)	15
Figure 13 : coded modulation result (same transmission energy)	16
Figure 14 : coded modulation result (same information energy)	17
Figure 15 : All Sechemes with same Transmission Energy	18
Figure 16 : All Sechemes with same Information Energy	18
Figure 17 : QPSK With Rayleigh flat fading Channel	20
Figure 19 : BER Results for QPSK Rayleigh Flat Fading	20
Figure 20 : QPSK With Frequency Selective Channel	21
Figure 21 : BER Results for QPSK Frequency Selective	21
Figure 22 : 16-QAM With Rayleigh flat fading Channel	22
Figure 23 : BER Results for 16-QAM With Rayleigh Channel	22
Figure 24 : 16-QAM With Frequency Selective Channel	23
Figure 25 : BER Results for 16-QAM Frequency Selective	23

Problem 1: Execution time of DFT and FFT

1. Results

Execution time (DFT) : 1.550020 seconds

Execution time (FFT) : 0.031023 seconds

FFT is faster than DFT by a factor of 49.96

Figure 1: FFT & DFT execution times

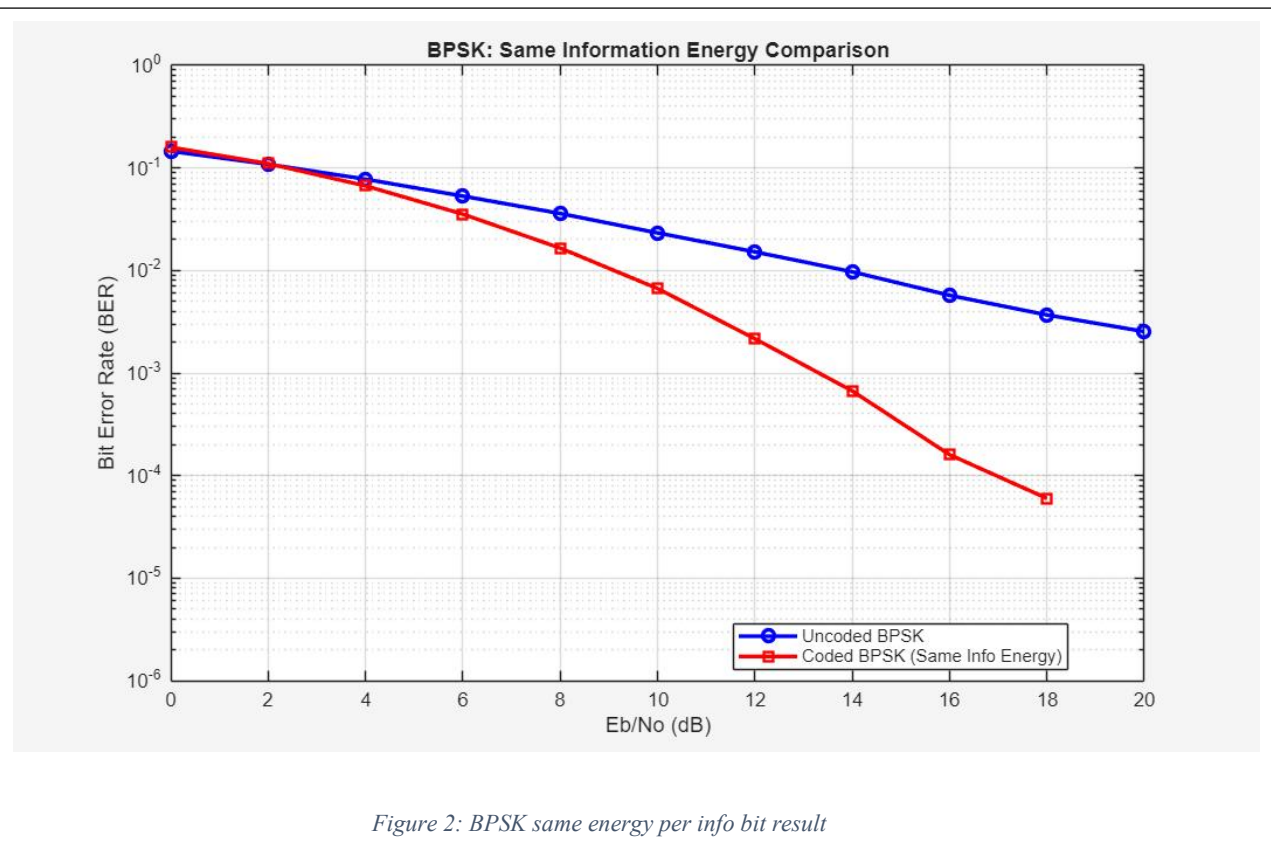
❖ Comment :

After simulating both DFT and FFT, we observe that FFT is much faster than DFT. This is because DFT computes all values directly, requiring N^2 operations, while FFT computes the same result more efficiently by splitting the problem into smaller parts and reusing calculations, requiring only $N \log N$ operations.

Problem 2: Bit-error rate performance for BPSK and 16-QAM over Rayleigh flat fading channel

1. BPSK

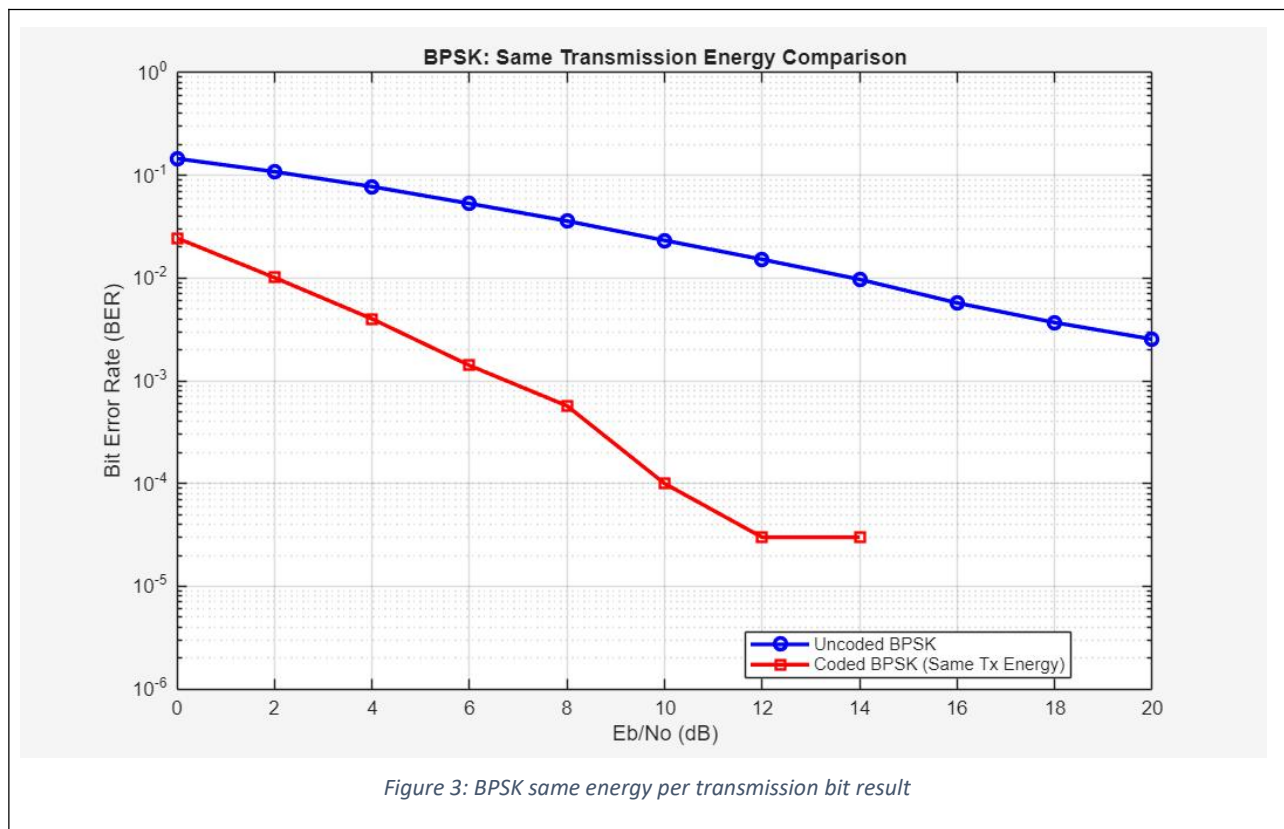
1.1. BPSK uncoded vs coded (same energy per information bit) Results



❖ Comment :

Both coded and uncoded BPSK show decreasing bit error rates as signal strength increases, but coded BPSK performs better than uncoded BPSK due to its error correction capabilities, achieving lower error rates at the same signal strength.

1.2. BPSK uncoded vs coded (same energy per transmission bit) Results



❖ Comment :

As signal strength increases, coded BPSK achieves much lower error rates than uncoded BPSK when using the same transmission energy.

1.3. BPSK with Same Transmission Energy Vs. BPSK with Same Information Energy

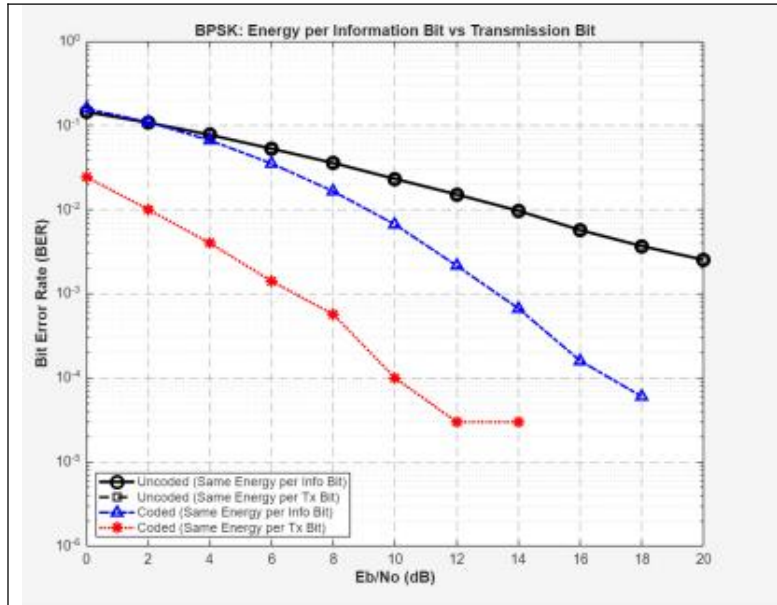


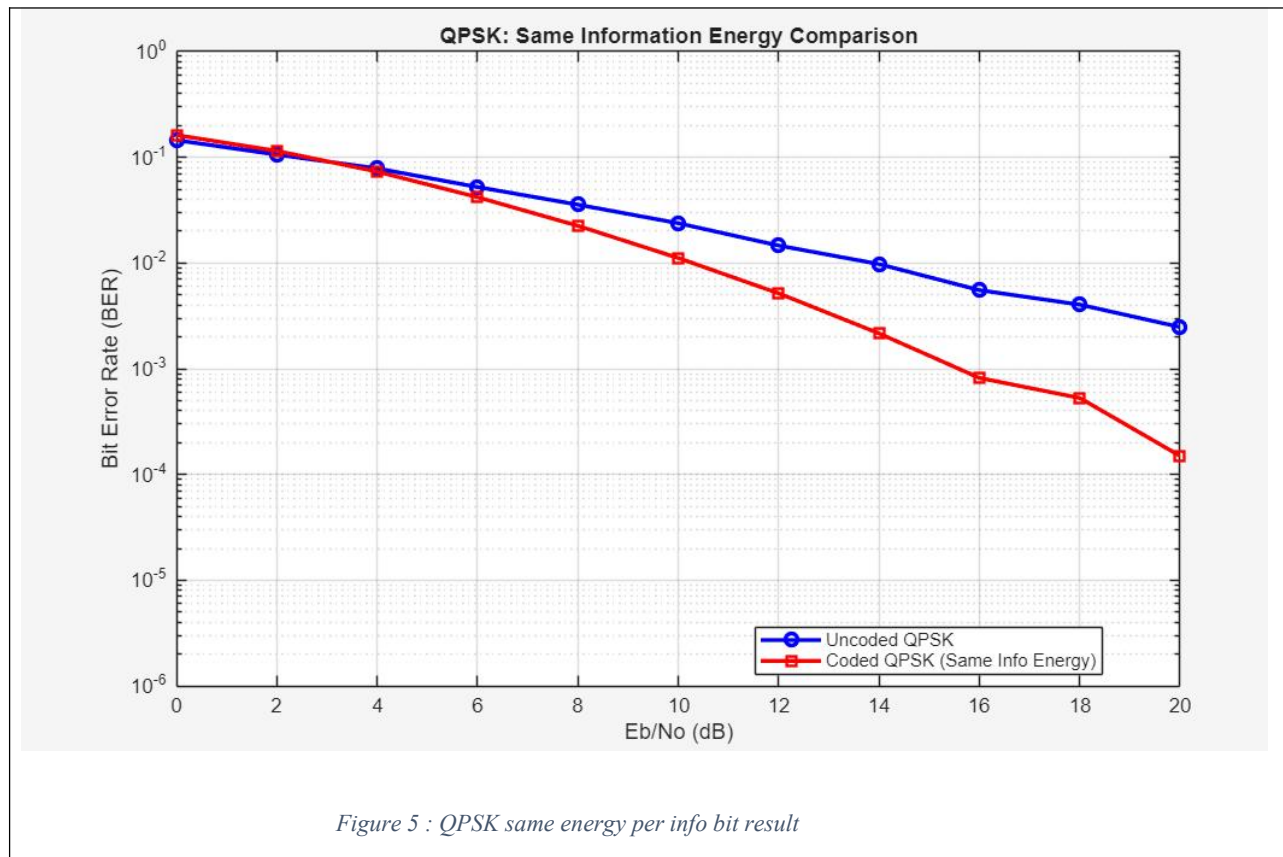
Figure 4 : BPSK Info vs. Transmission

❖ Comment :

For BPSK, the two uncoded lines (same energy per info bit and same energy per transmission bit) overlap completely and perform identically because without coding, information bits and transmission bits are the same. The coded scheme shows better performance, showing the strong advantage of coding.

2. QPSK

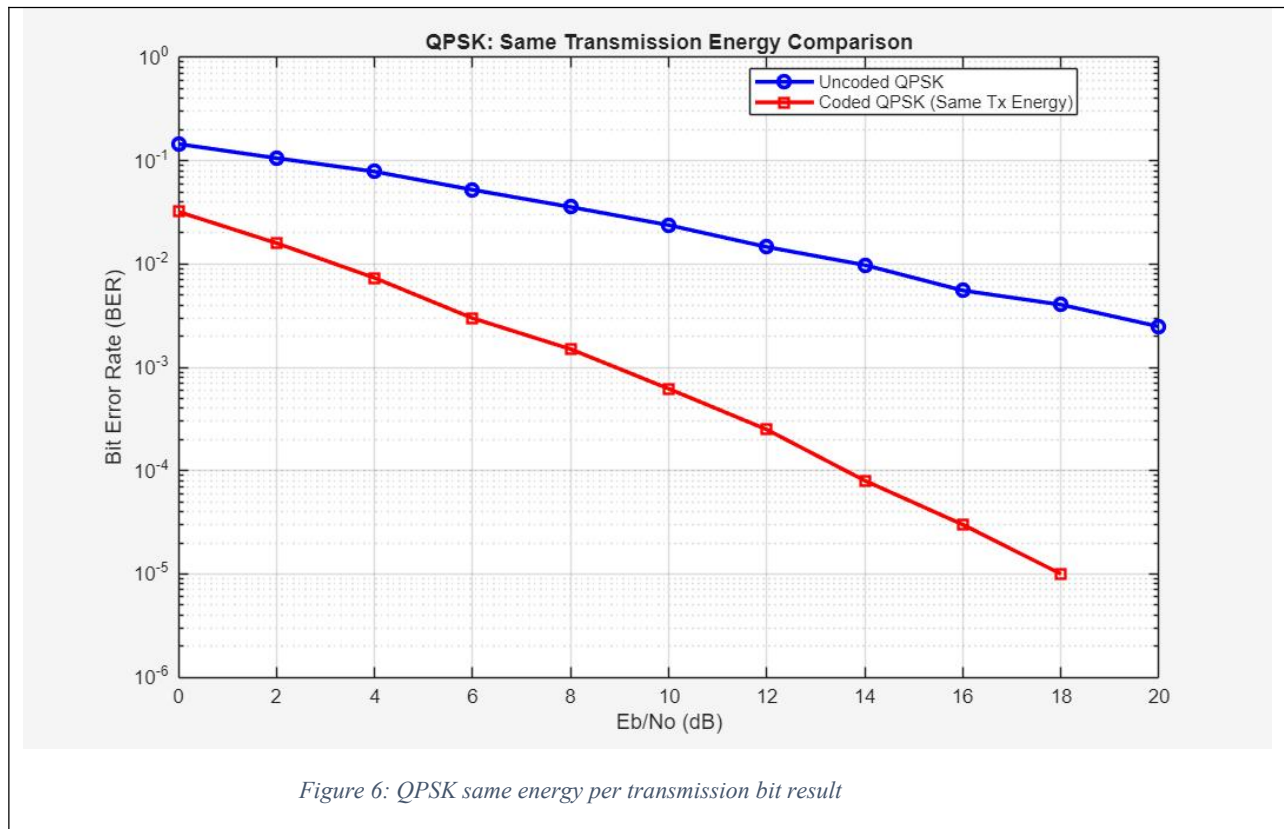
2.1. QPSK uncoded vs coded (same energy per information bit) Results



❖ Comment :

Coded QPSK and uncoded QPSK perform similarly at low signal strengths, but coded QPSK becomes better at higher signal strengths, achieving lower error rates when using the same information energy.

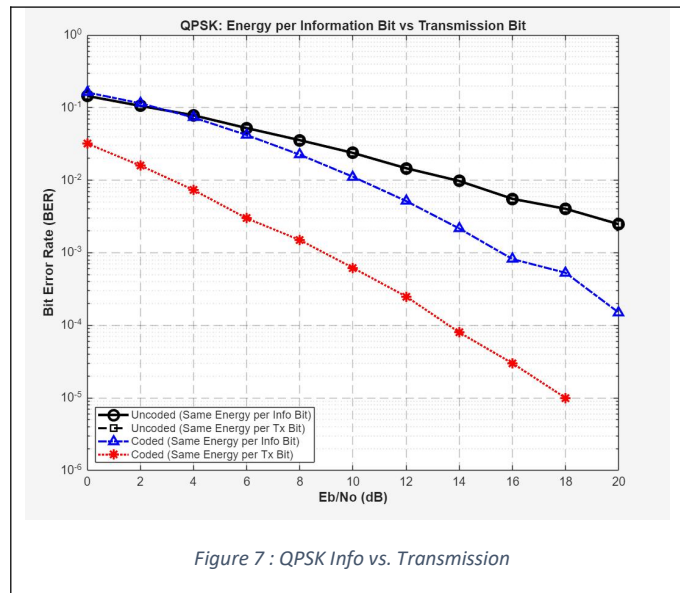
2.2. QPSK uncoded vs coded (same energy per transmission bit) Results



❖ Comment :

Coded QPSK performs much better than uncoded QPSK when using the same transmission energy.

2.3. QPSK with Same Transmission Energy Vs. QPSK with Same Information Energy

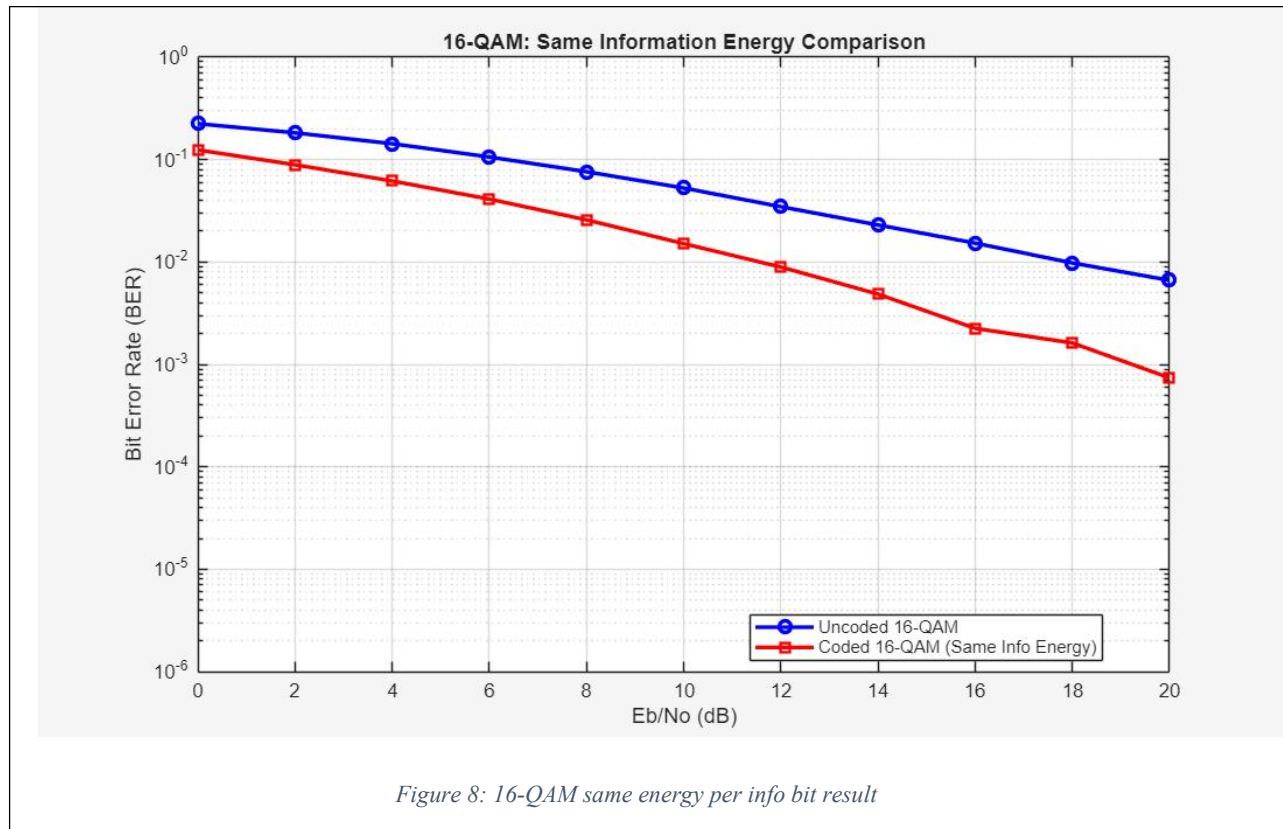


❖ Comment :

For QPSK, the two uncoded lines (same energy per info bit and same energy per transmission bit) also overlap and perform identically because without coding, there's no difference between information bits and transmission bits. The coded line shows better performance than the uncoded schemes, achieving much lower error rates.

3. 16-QAM

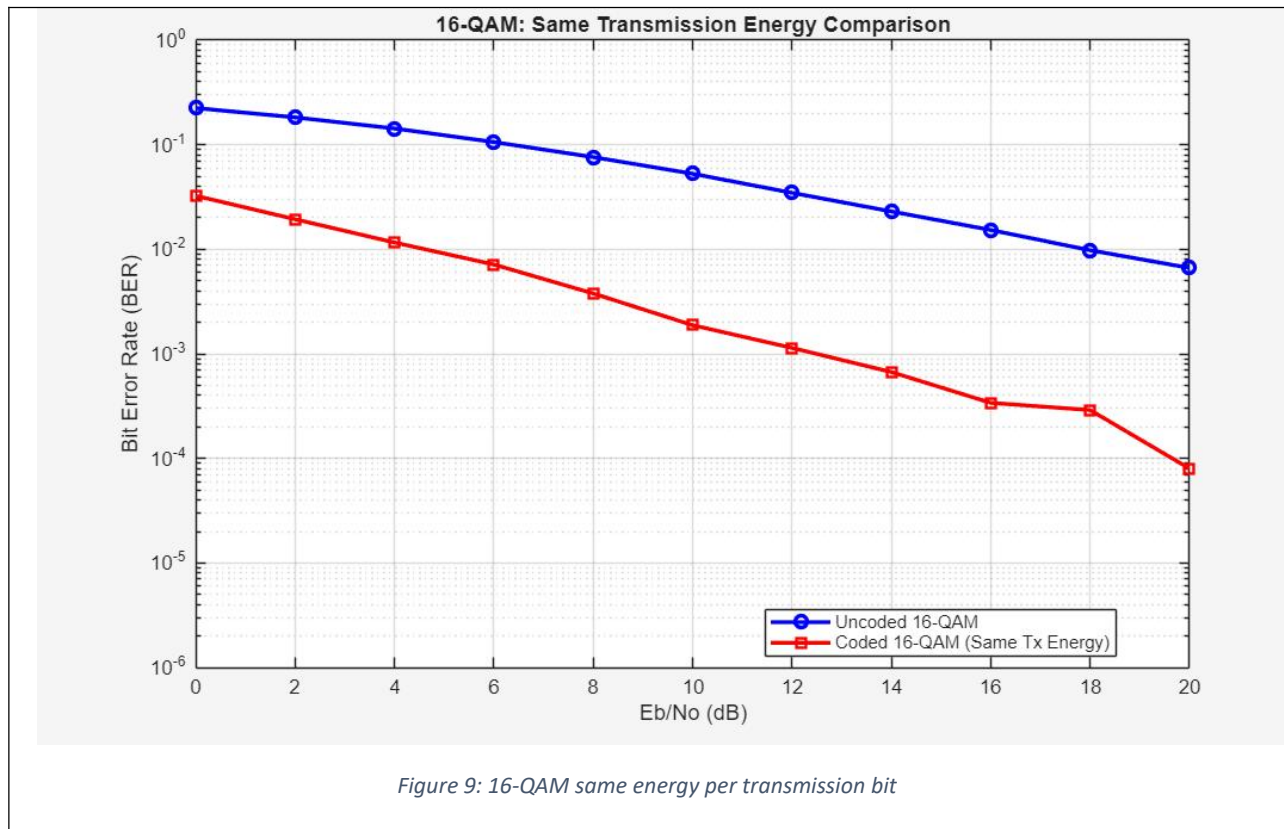
3.1. 16-QAM uncoded vs coded (same energy per information bit) Results



❖ Comment :

Coded 16-QAM performs better than uncoded 16-QAM at the same information energy, achieving lower error rates across all signal strengths.

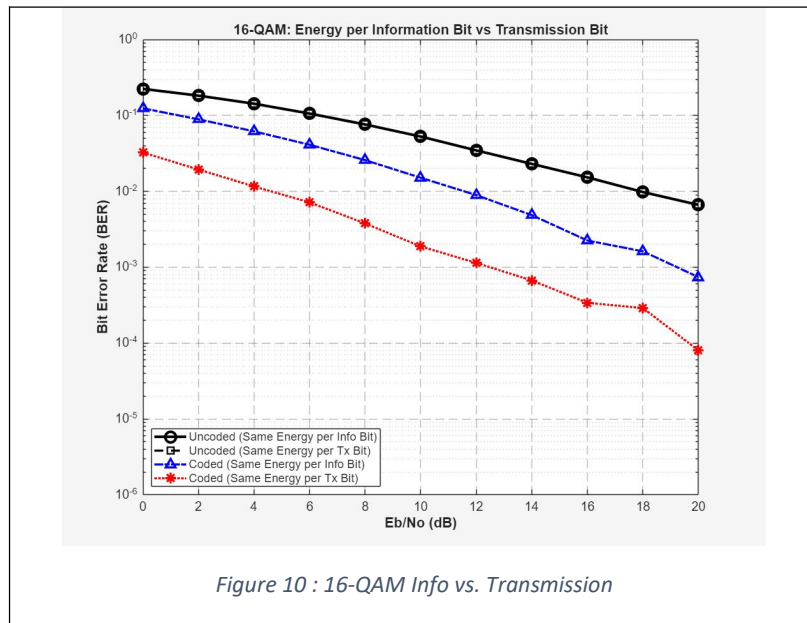
3.2. 16-QAM uncoded vs coded (same energy per transmission bit) Results



❖ Comment :

Coded 16-QAM performs better than uncoded 16-QAM when using the same transmission energy.

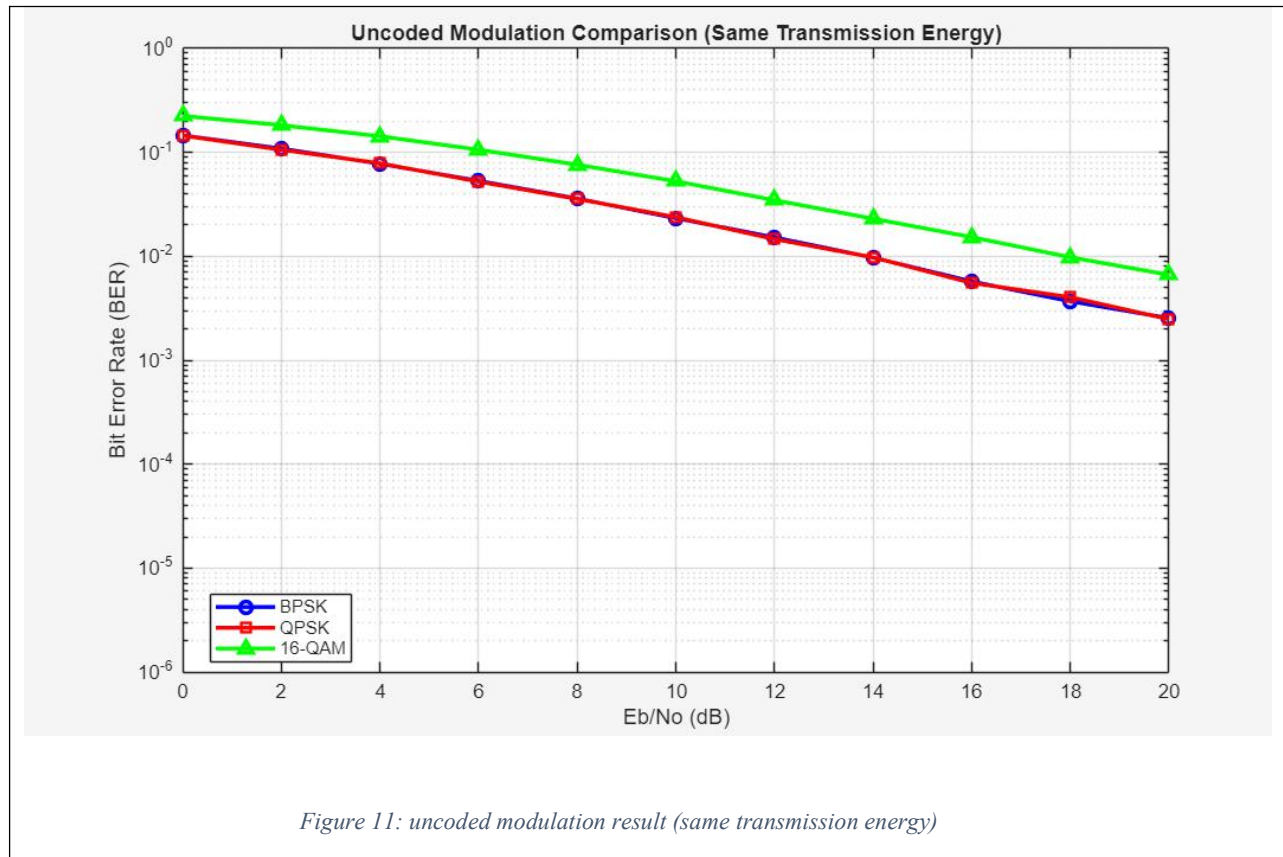
3.3. 16-QAM with Same Transmission Energy Vs. 16-QAM with Same Information Energy



❖ **Comment :**

For 16-QAM, the two uncoded lines (same energy per info bit and same energy per transmission bit) again overlap because uncoded schemes have no difference between information and transmission bits. The coded line performs best overall.

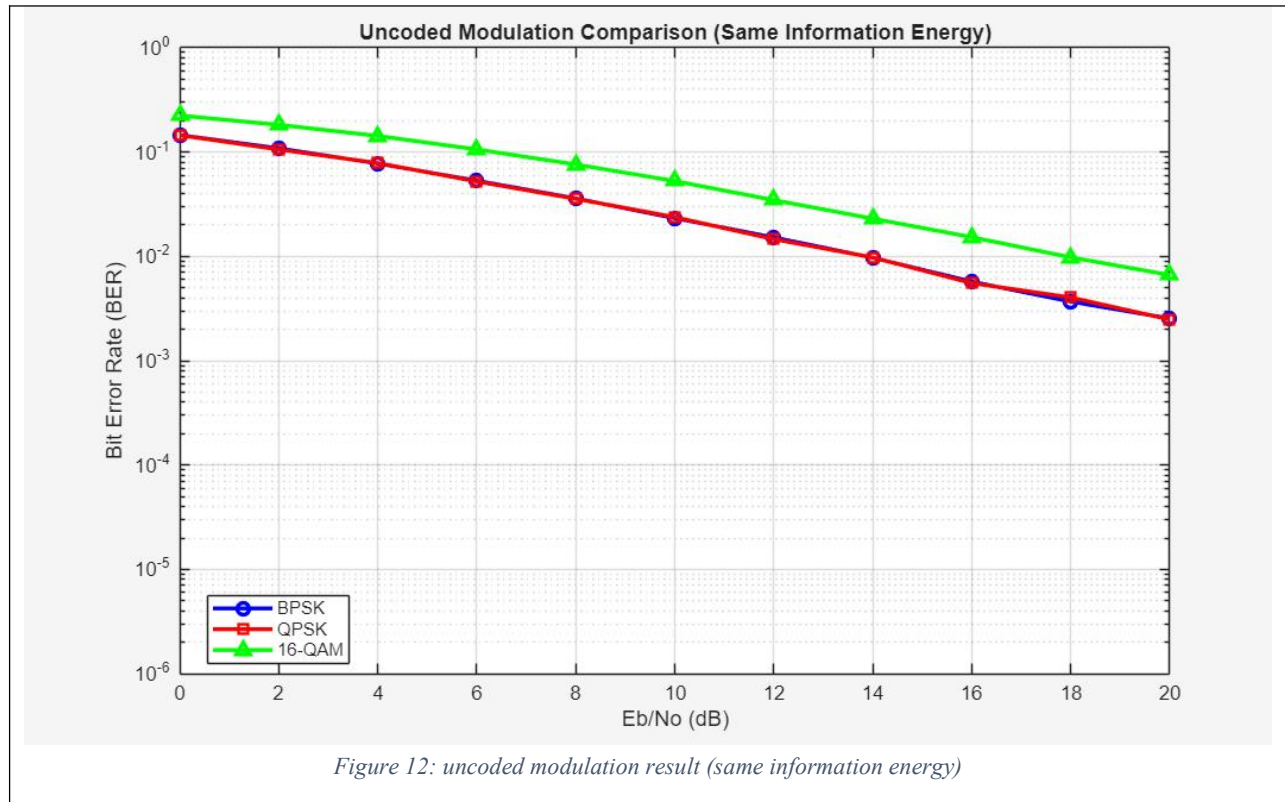
4. Uncoded Modulation Comparison (Same Transmission Energy) Results



❖ Comment :

BPSK performs best with the lowest error rates, QPSK shows slightly higher error rates, and 16-QAM has the highest error rates when all three use the same transmission energy.

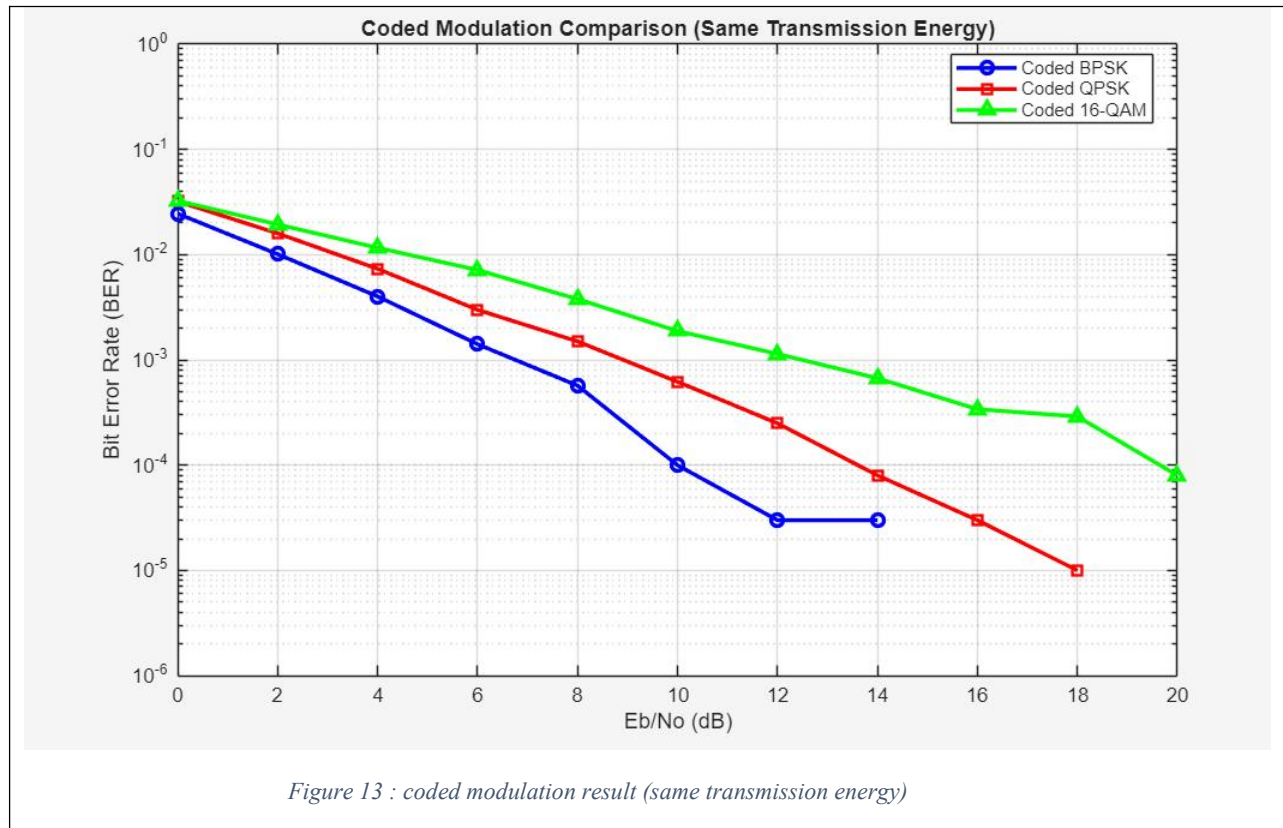
5. Uncoded Modulation Comparison (Same information Energy) Results



❖ Comment :

BPSK performs best with the lowest error rates, QPSK shows slightly higher error rates, and 16-QAM has the highest error rates when all three use the same information energy.

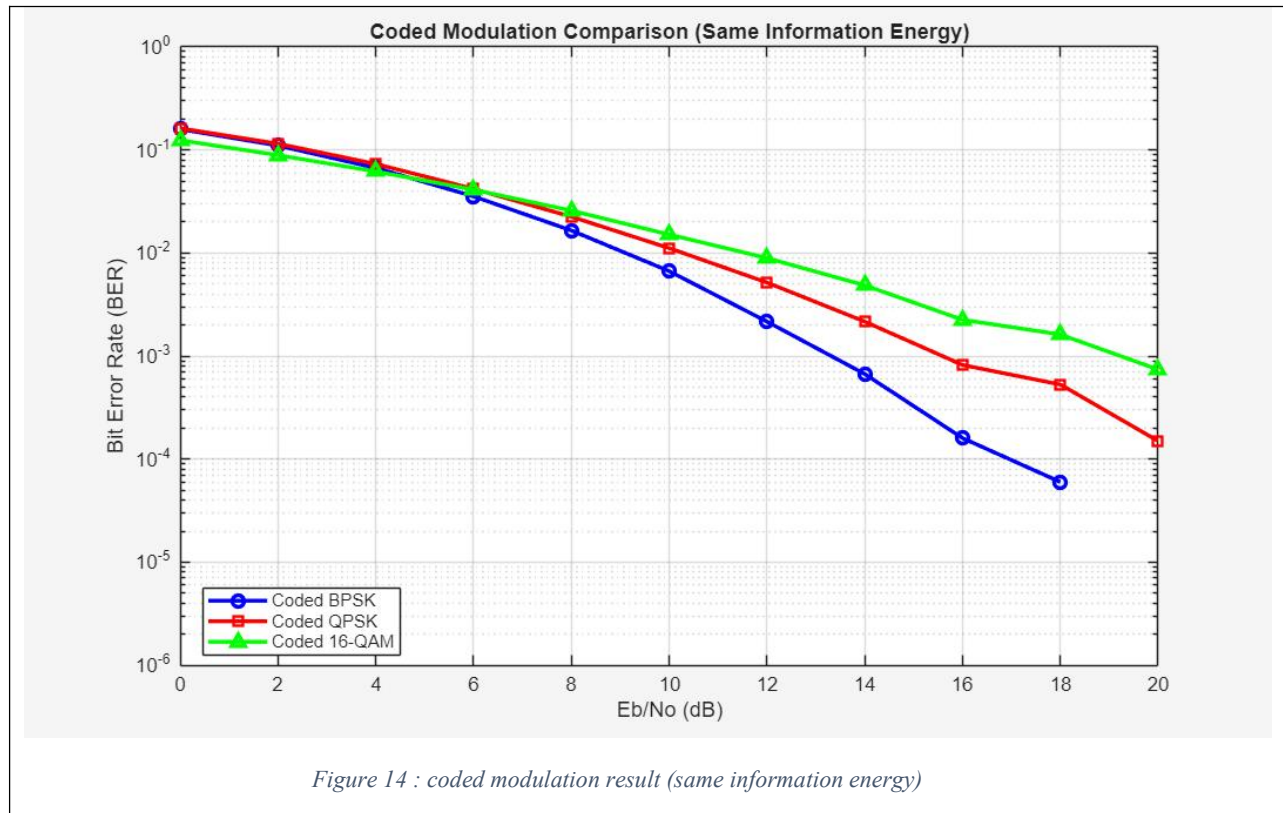
6. Coded Modulation Comparison (Same Transmission Energy) Results



❖ Comment :

Coded BPSK performs best with the lowest error rates, coded QPSK is in the middle, and coded 16-QAM has the highest error rates when all use the same transmission energy, showing that simpler modulation schemes benefit more from coding and achieve better reliability.

7. Coded Modulation Comparison (Same information Energy) Results



❖ Comment :

When using the same information energy, coded BPSK performs best with the lowest error rates, coded QPSK is slightly worse, and coded 16-QAM has the highest error rates.

8. Coded Modulation Comparison (Same information Energy) Results

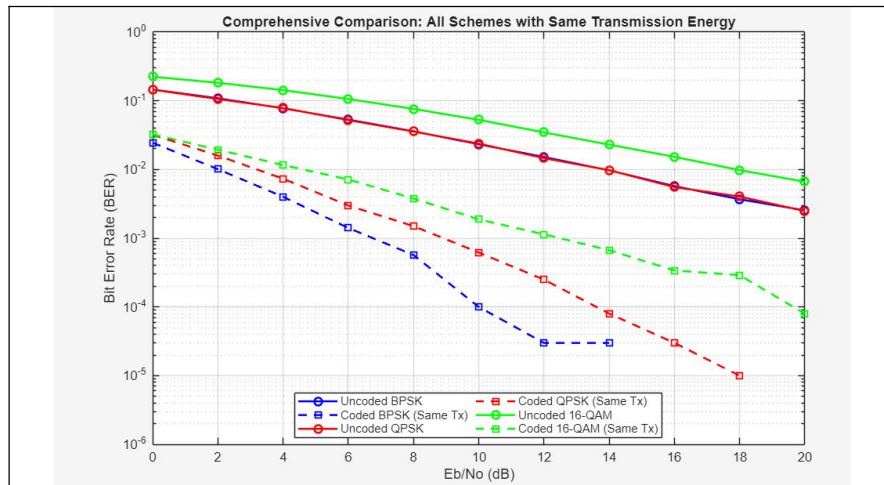


Figure 15 : All Sechemes with same Transmission Energy

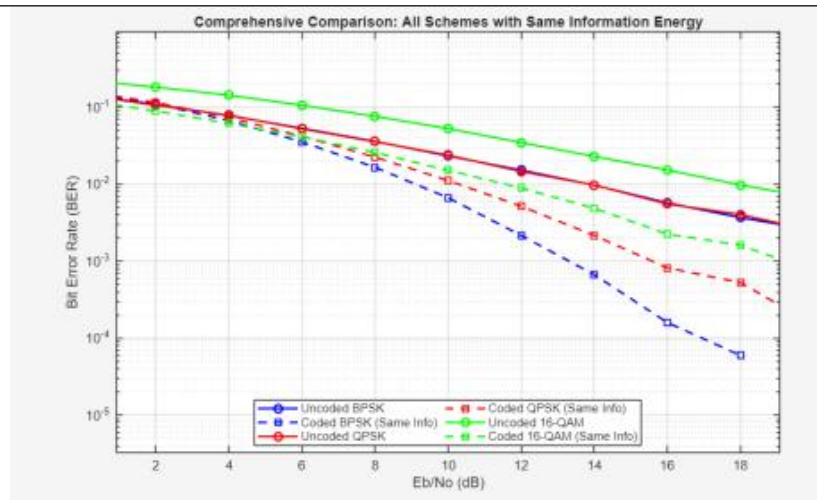


Figure 16 : All Sechemes with same Information Energy

❖ Comment :

- 1) **Figure 15 (Same Transmission Energy):** When using the same transmission energy, coded schemes still achieve better performance than uncoded schemes . Coded BPSK achieves the lowest error rates, while uncoded 16-QAM performs the worst.
- 2) **Figure 16 (Same Information Energy):** When compared at the same information energy, coded schemes achieve better performance than uncoded schemes but their performance is less than that of Figure 15. Among the coded schemes, coded BPSK shows the best performance, while coded 16-QAM shows the worst. All coded schemes achieve much lower error rates than their uncoded form, demonstrating the effectiveness of error-correction coding.

❖ **Comparison between 2 graphs :**

The main difference is that Figure 15 compares schemes based on actual data transmitted (information energy), while Figure 16 compares them based on total power used (transmission energy). Figure 16 shows a much bigger advantage for coded schemes because coding achieves better performance without needing extra transmission power, making it more efficient in real-world applications.

Problem 3: OFDM System Simulation

1. QPSK With Rayleigh Flat Fading Channel

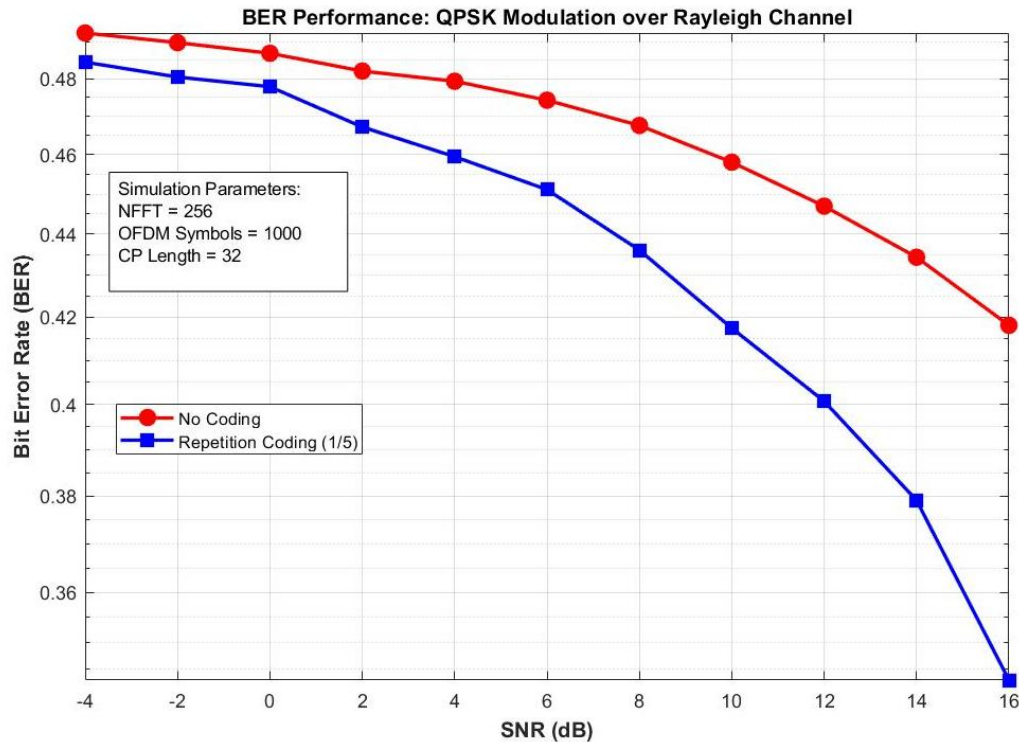


Figure 17 : QPSK With Rayleigh flat fading Channel

❖ Comment :

- 1) The graph shows that repetition coding (1/5) consistently reduces the BER compared to no coding, indicating improved reliability.
- 2) The performance gain is more noticeable at low SNR values, where noise and fading effects are stronger.
- 3) At higher SNR, the gap between the curves decreases since the channel conditions improve. However, this improvement comes at the cost of reduced data rate due to the added redundancy.

The performance is still poor with high BER across all SNR, but repetition coding shows a consistent, though modest, improvement as SNR increases. This suggests the Rayleigh fading is very severe, and simple repetition coding alone is insufficient for reliable communication.

===== BER Results =====			
SNR (dB)	No Coding	Repetition (1/5)	Improvement
-4	4.92e-01	4.84e-01	1.02
-2	4.90e-01	4.80e-01	1.02
0	4.87e-01	4.78e-01	1.02
2	4.82e-01	4.67e-01	1.03
4	4.79e-01	4.60e-01	1.04
6	4.74e-01	4.51e-01	1.05
8	4.68e-01	4.36e-01	1.07
10	4.58e-01	4.17e-01	1.10
12	4.47e-01	4.01e-01	1.12
14	4.34e-01	3.79e-01	1.15
16	4.18e-01	3.43e-01	1.22
=====			

Figure 19 : BER Results for QPSK Rayleigh Flat Fading

2. QPSK With Frequency Selective Channel

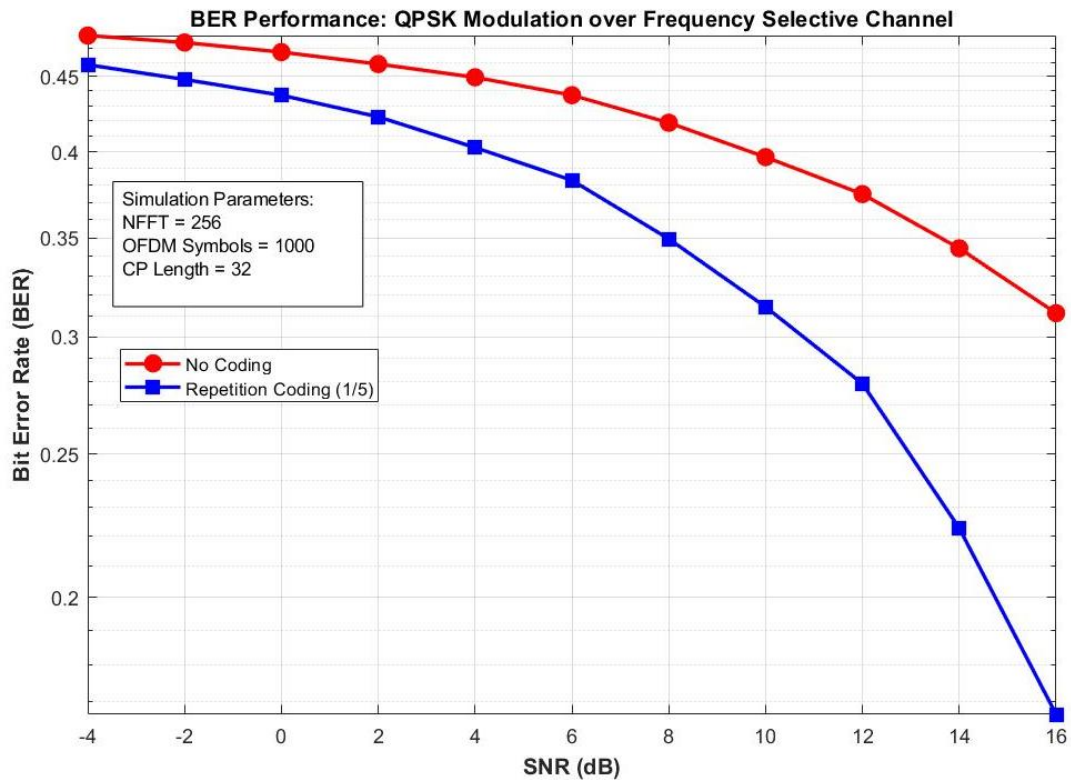


Figure 20 : QPSK With Frequency Selective Channel

❖ Comment :

- 1) The graph highlights that frequency selective fading is much more severe than Rayleigh flat fading, resulting in higher BER overall.
- 2) Repetition coding provides a significant improvement, with a large gap between coded and uncoded curves, especially at moderate SNR values. This is because frequency selective fading causes deep spectral nulls that strongly degrade uncoded data. By adding redundancy, repetition coding helps recover information despite these deep fades.

The system is still failing with BER near 0.5 at low SNR, but repetition coding shows measurable improvement at higher SNR values. This indicates that coding starts helping as signal strength increases, but the frequency selective channel remains extremely challenging.

===== BER Results =====			
SNR (dB)	No Coding	Repetition (1/5)	Improvement
-4	4.80e-01	4.58e-01	1.05
-2	4.74e-01	4.48e-01	1.06
0	4.67e-01	4.37e-01	1.07
2	4.59e-01	4.23e-01	1.09
4	4.49e-01	4.03e-01	1.12
6	4.37e-01	3.83e-01	1.14
8	4.19e-01	3.49e-01	1.20
10	3.97e-01	3.14e-01	1.26
12	3.75e-01	2.79e-01	1.34
14	3.44e-01	2.23e-01	1.55
16	3.11e-01	1.67e-01	1.87
=====			

Figure 21 : BER Results for QPSK Frequency Selective

3. 16-QAM With Rayleigh Flat Fading Channel

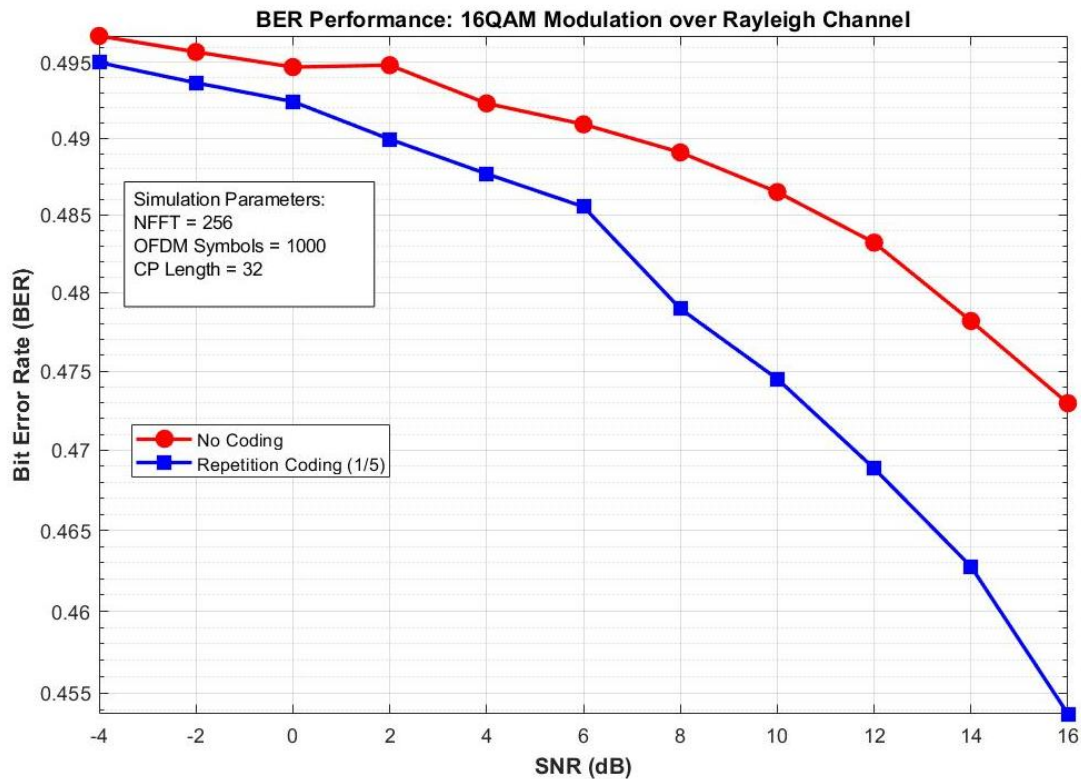


Figure 22 : 16-QAM With Rayleigh flat fading Channel

❖ Comment :

Repetition coding significantly improves reliability in challenging channels, especially for QPSK in frequency-selective conditions. Higher-order modulation like 16QAM is more sensitive to channel impairments, requiring better SNR for acceptable performance. The trade-off is always between data rate (higher with 16QAM, lower with coding) and reliability (higher with QPSK and coding).

The BER is near 0.5 for all SNR values, meaning the system is essentially failing regardless of coding. Repetition coding provides almost no improvement here, showing it cannot overcome such extreme channel conditions.

===== BER Results =====			
SNR (dB)	No Coding	Repetition (1/5)	Improvement
-4	4.97e-01	4.95e-01	1.00
-2	4.96e-01	4.94e-01	1.00
0	4.95e-01	4.92e-01	1.00
2	4.95e-01	4.90e-01	1.01
4	4.92e-01	4.88e-01	1.01
6	4.91e-01	4.86e-01	1.01
8	4.89e-01	4.79e-01	1.02
10	4.86e-01	4.75e-01	1.03
12	4.83e-01	4.69e-01	1.03
14	4.78e-01	4.63e-01	1.03
16	4.73e-01	4.54e-01	1.04
=====			

Figure 23 : BER Results for 16-QAM With Rayleigh Channel

4. 16-QAM With Frequency Selective Channel

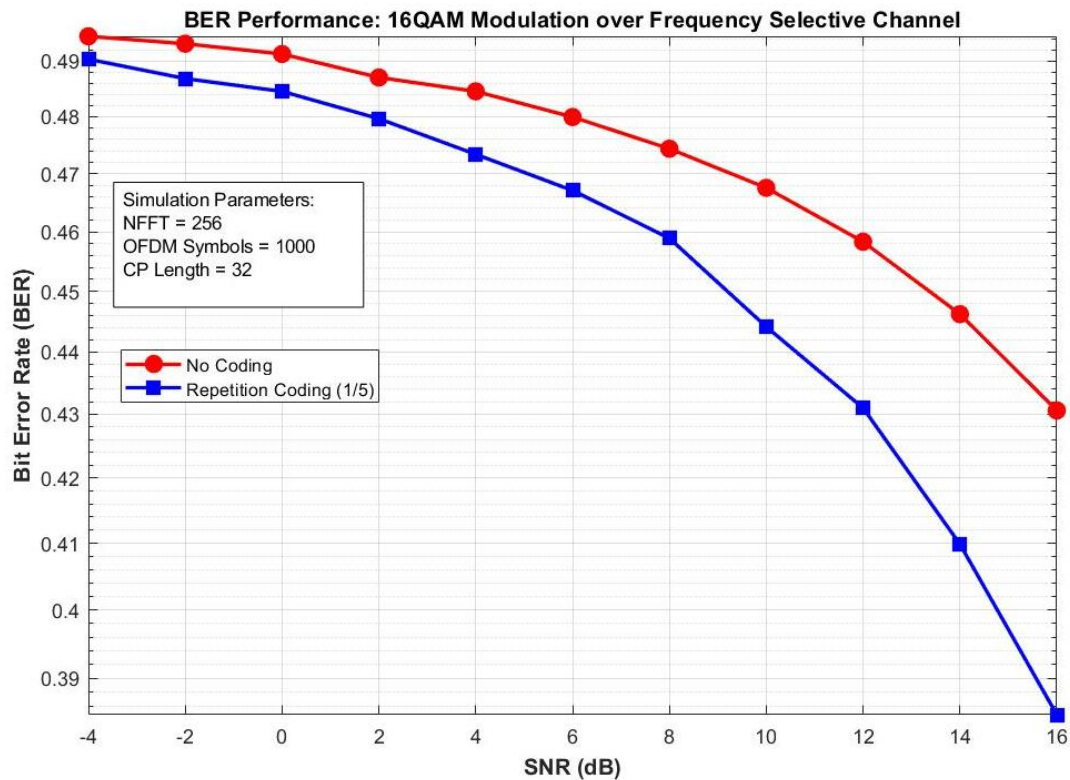


Figure 24 : 16-QAM With Frequency Selective Channel

❖ Comment :

This is the worst-case scenario: combining 16QAM's sensitivity with frequency selective fading's distortions. Both systems fail completely at low SNR (BER ≈ 0.5), essentially random guessing. Only at very high SNR (above 10 dB) does repetition coding provide marginal improvement, but overall performance remains extremely poor. This clearly shows why adaptive modulation is essential in practice—16QAM should only be used in very favorable channel conditions.

The system is failing with BER near 0.5, essentially random guessing, across the entire SNR range. Repetition coding provides only a slight improvement, confirming that 16QAM is too fragile for frequency selective channels without much stronger coding.

===== BER Results =====			
SNR (dB)	No Coding	Repetition (1/5)	Improvement
-4	4.94e-01	4.90e-01	1.01
-2	4.93e-01	4.87e-01	1.01
0	4.91e-01	4.85e-01	1.01
2	4.87e-01	4.80e-01	1.02
4	4.85e-01	4.73e-01	1.02
6	4.80e-01	4.67e-01	1.03
8	4.74e-01	4.59e-01	1.03
10	4.68e-01	4.44e-01	1.05
12	4.58e-01	4.31e-01	1.06
14	4.46e-01	4.10e-01	1.09
16	4.31e-01	3.85e-01	1.12
=====			

Figure 25 : BER Results for 16-QAM Frequency Selective

Appendix : Full Matlab code

```
%% =====choose which problem to
run=====%%
clc; clear; close all;

choice = menu('Choose Problem to Run:', ...
'Problem 1:Execution time of DFT and FFT', ...
'Problem 2: Bit-error rate performance for BPSK and 16-QAM over Rayleigh
flat fading channel', ...
'Problem 3: OFDM System Simulation');

switch choice
case 1
problem_1(); % Runs your FULL Problem 1 code
case 2
problem_2(); % Runs your FULL Problem 2 code
case 3
problem_3();
otherwise
disp('No selection made.');
```

```
end

%% ===== Problem 1
code=====%%
function problem_1 ()
%% Generate test signal
L = 4096; % Signal length
xi = randn(1, L); % randomize signal x
%% Compare execution time between custom DFT and MATLAB FFT implementation

% Time for custom DFT function
tic; % Start stopwatch timer
X_dft = DFT(xi); % Call custom DFT function to compute spectrum
time_dft = toc; % Stop timer and store elapsed time

% Time for MATLAB built-in FFT
tic; % Start stopwatch timer
X_fft = fft(xi); % Compute FFT using MATLAB's optimized algorithm
time_fft = toc; % Stop timer and store elapsed time

% Display execution time results
fprintf('Execution time (DFT) : %f seconds\n', time_dft);
fprintf('Execution time (FFT) : %f seconds\n', time_fft);

%% Custom DFT Function
function X = DFT(x)

N = length(x); % Length of input signal
X = zeros(1, N); % Initialize output array for DFT results

% Nested loops to compute DFT manually (O(N^2) complexity)
for k = 1:N % Loop over frequency bins
for n = 1:N % Loop over time samples
% Accumulate contribution of each sample
X(k) = X(k) + x(n) * exp(-1j * 2 * pi * (n-1) * (k-1) / N);
```

```

end
end
end
%% Compare Performance
fprintf('\nFFT is faster than DFT by a factor of %.2f\n', time_dft /
time_fft);

end
%% ===== Problem 2
code=====%%
function problem_2 ()
% Common parameters
numBits = 1e5; % Number of bits per SNR point
Eb = 1; % Energy per bit
EbN0_dB = 0:2:20; % Eb/No range in dB
EbN0 = 10.^(EbN0_dB/10); % Linear Eb/No

% Repetition code parameters
code_rate = 1/5; % Rate 1/5 repetition code

%% 1. BPSK Simulation (Uncoded vs Coded)
fprintf('Simulating BPSK...\n');

% Initialize BER arrays
BER_BPSK_uncoded = zeros(size(EbN0));
BER_BPSK_coded = zeros(size(EbN0));
BER_BPSK_coded_sameInfo = zeros(size(EbN0));
BER_BPSK_coded_sameTx = zeros(size(EbN0));

for snrIdx = 1:length(EbN0)
% Current Eb/No
EbN0_current = EbN0(snrIdx);
% Noise variance
No = Eb / EbN0_current;
sigma = sqrt(No/2);
% Generate random bits
b_k = randi([0 1], 1, numBits);
%% Uncoded BPSK
% BPSK modulation: 0 -> +√Eb, 1 -> -√Eb
x_k = (1 - 2*b_k) * sqrt(Eb);
% Rayleigh fading channel
h_r = randn(1, numBits) * sqrt(1/2);
h_i = randn(1, numBits) * sqrt(1/2);
h = h_r + 1j*h_i;
% AWGN
n_c = randn(1, numBits) * sigma;
n_s = randn(1, numBits) * sigma;
n = n_c + 1j*n_s;
% Received signal
y_k = h .* x_k + n;
% Channel compensation (assuming perfect CSI)
x_hat = y_k ./ h;
% Decision: real part > 0 -> 0, real part < 0 -> 1
b_hat = real(x_hat) < 0;
% BER calculation
BER_BPSK_uncoded(snrIdx) = sum(b_hat ~= b_k) / numBits;
%% Coded BPSK (Rate 1/5 Repetition Code)

```

```

% Encode: repeat each bit 5 times
b_coded = repelem(b_k, 5);
numCodedBits = length(b_coded);
% BPSK modulation for coded bits
x_coded = (1 - 2*b_coded) * sqrt(Eb); % Same symbol energy
% Rayleigh fading for coded symbols
h_coded = (randn(1, numCodedBits) + 1j*randn(1, numCodedBits)) * sqrt(1/2);
% AWGN for coded symbols
n_coded = (randn(1, numCodedBits) + 1j*randn(1, numCodedBits)) * sigma;
% Received coded signal
y_coded = h_coded .* x_coded + n_coded;
% Channel compensation
x_coded_hat = y_coded ./ h_coded;
% Hard decision on coded symbols
b_coded_hat = real(x_coded_hat) < 0;
% Decode: majority voting for every 5 bits
b_decoded = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat((i-1)*5+1:i*5);
    b_decoded(i) = sum(segment) >= 3; % Majority vote
end
% BER calculation
BER_BPSK_coded(snrIdx) = sum(b_decoded ~= b_k) / numBits;
%% Coded BPSK with Same Information Energy
% Energy per coded symbol = Eb * code_rate (since more symbols for same
info)
x_coded_sameInfo = (1 - 2*b_coded) * sqrt(Eb * code_rate);
% Channel and noise (same as above)
y_coded_sameInfo = h_coded .* x_coded_sameInfo + n_coded;
x_coded_hat_sameInfo = y_coded_sameInfo ./ h_coded;
b_coded_hat_sameInfo = real(x_coded_hat_sameInfo) < 0;
% Decode
b_decoded_sameInfo = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat_sameInfo((i-1)*5+1:i*5);
    b_decoded_sameInfo(i) = sum(segment) >= 3;
end
BER_BPSK_coded_sameInfo(snrIdx) = sum(b_decoded_sameInfo ~= b_k) / numBits;
%% Coded BPSK with Same Transmission Energy
% Energy per bit is reduced for coded system to maintain same total energy
Eb_coded = Eb * code_rate;
No_coded = Eb_coded / EbN0_current;
sigma_coded = sqrt(No_coded/2);
x_coded_sameTx = (1 - 2*b_coded) * sqrt(Eb_coded);
n_coded_sameTx = (randn(1, numCodedBits) + 1j*randn(1, numCodedBits)) *
sigma_coded;
y_coded_sameTx = h_coded .* x_coded_sameTx + n_coded_sameTx;
x_coded_hat_sameTx = y_coded_sameTx ./ h_coded;
b_coded_hat_sameTx = real(x_coded_hat_sameTx) < 0;
% Decode
b_decoded_sameTx = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat_sameTx((i-1)*5+1:i*5);
    b_decoded_sameTx(i) = sum(segment) >= 3;
end
BER_BPSK_coded_sameTx(snrIdx) = sum(b_decoded_sameTx ~= b_k) / numBits;
end

```

```

%% 2. QPSK Simulation (Uncoded vs Coded) - CORRECTED
fprintf('Simulating QPSK...\n');

% Initialize BER arrays
BER_QPSK_uncoded = zeros(size(EbN0));
BER_QPSK_coded = zeros(size(EbN0));
BER_QPSK_coded_sameInfo = zeros(size(EbN0));
BER_QPSK_coded_sameTx = zeros(size(EbN0));

% QPSK parameters
M_QPSK = 4;
k_QPSK = log2(M_QPSK); % 2 bits per symbol
Es_QPSK = k_QPSK * Eb; % Energy per symbol = 2*Eb

% QPSK constellation with Gray coding (normalized to unit average power)
% Gray mapping: 00->(1+1j), 01->(1-1j), 11->(-1-1j), 10->(-1+1j)
QPSK_const_normalized = [1+1j, 1-1j, -1-1j, -1+1j] / sqrt(2); %
Normalization factor sqrt(2)

% Gray mapping indices
gray_map_QPSK = [0, 1, 3, 2] + 1; % Gray mapping: 00->1, 01->2, 11->3, 10->4

for snrIdx = 1:length(EbN0)
    EbN0_current = EbN0(snrIdx);
    No = Eb / EbN0_current;
    sigma = sqrt(No/2);
    % Generate random bits (must be multiple of 2 for QPSK)
    b_k = randi([0 1], 1, numBits);
    %% Uncoded QPSK
    numSymbols = numBits / 2;
    % Reshape bits into pairs
    bits_resaped = reshape(b_k, 2, numSymbols)';
    % Modulate QPSK symbols
    symbols = zeros(numSymbols, 1);
    for i = 1:numSymbols
        bits = bits_resaped(i, :);
        % Convert 2 bits to decimal (0-3)
        dec_val = bits(1)*2 + bits(2);
        % Apply Gray mapping
        gray_idx = gray_map_QPSK(dec_val + 1);
        % Get constellation point and scale by sqrt(Es)
        symbols(i) = QPSK_const_normalized(gray_idx) * sqrt(Es_QPSK);
    end
    % Rayleigh fading channel
    h = (randn(numSymbols, 1) + 1j*randn(numSymbols, 1)) * sqrt(1/2);
    % AWGN
    n = (randn(numSymbols, 1) + 1j*randn(numSymbols, 1)) * sigma;
    % Received signal
    y = h .* symbols + n;
    % Channel compensation (perfect CSI)
    symbols_hat = y ./ h;
    % Demodulation (minimum distance)
    b_hat = zeros(1, numBits);
    for i = 1:numSymbols
        % Normalize received symbol
        r = symbols_hat(i) / sqrt(Es_QPSK);

```

```

% Find closest constellation point
distances = zeros(1, 4);
for j = 1:4
distances(j) = abs(r - QPSK_const_normalized(j));
end
[~, min_idx] = min(distances);
% Reverse Gray mapping to get bits
const_idx = find(gray_map_QPSK == min_idx) - 1;
% Convert to bits (MSB first)
b_hat((i-1)*2+1:i*2) = [bitget(const_idx, 2), bitget(const_idx, 1)];
end
BER_QPSK_uncoded(snrIdx) = sum(b_hat ~= b_k) / numBits;
%% Coded QPSK (Rate 1/5 Repetition Code)
% Encode each bit with repetition code
b_coded = repelem(b_k, 5);
numCodedBits = length(b_coded);
numCodedSymbols = numCodedBits / 2;
% Reshape coded bits into pairs
b_coded_reshaped = reshape(b_coded, 2, numCodedSymbols)';
% Modulate coded bits
symbols_coded = zeros(numCodedSymbols, 1);
for i = 1:numCodedSymbols
bits = b_coded_reshaped(i, :);
dec_val = bits(1)*2 + bits(2);
gray_idx = gray_map_QPSK(dec_val + 1);
symbols_coded(i) = QPSK_const_normalized(gray_idx) * sqrt(Es_QPSK);
end
% Rayleigh fading for coded symbols
h_coded = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1)) *
sqrt(1/2);
% AWGN for coded symbols
n_coded = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1)) *
sigma;
% Received coded signal
y_coded = h_coded .* symbols_coded + n_coded;
% Channel compensation
symbols_coded_hat = y_coded ./ h_coded;
% Demodulate coded symbols
b_coded_hat = zeros(1, numCodedBits);
for i = 1:numCodedSymbols
r = symbols_coded_hat(i) / sqrt(Es_QPSK);
distances = zeros(1, 4);
for j = 1:4
distances(j) = abs(r - QPSK_const_normalized(j));
end
[~, min_idx] = min(distances);
const_idx = find(gray_map_QPSK == min_idx) - 1;
b_coded_hat((i-1)*2+1:i*2) = [bitget(const_idx, 2), bitget(const_idx, 1)];
end
% Decode: majority voting for every 5 bits
b_decoded = zeros(1, numBits);
for i = 1:numBits
segment = b_coded_hat((i-1)*5+1:i*5);
b_decoded(i) = sum(segment) >= 3;
end
BER_QPSK_coded(snrIdx) = sum(b_decoded ~= b_k) / numBits;
%% Coded QPSK with Same Information Energy

```

```

% Reduce symbol energy by code rate
symbols_coded_sameInfo = symbols_coded * sqrt(code_rate);
y_coded_sameInfo = h_coded .* symbols_coded_sameInfo + n_coded;
symbols_coded_hat_sameInfo = y_coded_sameInfo ./ h_coded;
b_coded_hat_sameInfo = zeros(1, numCodedBits);
for i = 1:numCodedSymbols
    r = symbols_coded_hat_sameInfo(i) / sqrt(Es_QPSK * code_rate);
    distances = zeros(1, 4);
    for j = 1:4
        distances(j) = abs(r - QPSK_const_normalized(j));
    end
    [~, min_idx] = min(distances);
    const_idx = find(gray_map_QPSK == min_idx) - 1;
    b_coded_hat_sameInfo((i-1)*2+1:i*2) = [bitget(const_idx, 2),
    bitget(const_idx, 1)];
end
% Decode
b_decoded_sameInfo = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat_sameInfo((i-1)*5+1:i*5);
    b_decoded_sameInfo(i) = sum(segment) >= 3;
end
BER_QPSK_coded_sameInfo(snrIdx) = sum(b_decoded_sameInfo ~= b_k) / numBits;
%% Coded QPSK with Same Transmission Energy
Eb_coded = Eb * code_rate;
No_coded = Eb_coded / EbN0_current;
sigma_coded = sqrt(No_coded/2);
symbols_coded_sameTx = symbols_coded * sqrt(code_rate);
n_coded_sameTx = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1))
* sigma_coded;
y_coded_sameTx = h_coded .* symbols_coded_sameTx + n_coded_sameTx;
symbols_coded_hat_sameTx = y_coded_sameTx ./ h_coded;
b_coded_hat_sameTx = zeros(1, numCodedBits);
for i = 1:numCodedSymbols
    r = symbols_coded_hat_sameTx(i) / sqrt(Es_QPSK * code_rate);
    distances = zeros(1, 4);
    for j = 1:4
        distances(j) = abs(r - QPSK_const_normalized(j));
    end
    [~, min_idx] = min(distances);
    const_idx = find(gray_map_QPSK == min_idx) - 1;
    b_coded_hat_sameTx((i-1)*2+1:i*2) = [bitget(const_idx, 2), bitget(const_idx,
    1)];
end
% Decode
b_decoded_sameTx = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat_sameTx((i-1)*5+1:i*5);
    b_decoded_sameTx(i) = sum(segment) >= 3;
end
BER_QPSK_coded_sameTx(snrIdx) = sum(b_decoded_sameTx ~= b_k) / numBits;
end

%% 3. 16-QAM Simulation (Uncoded vs Coded)
fprintf('Simulating 16-QAM...\n');

% Initialize BER arrays

```

```

BER_16QAM_uncoded = zeros(size(EbN0));
BER_16QAM_coded = zeros(size(EbN0));
BER_16QAM_coded_sameInfo = zeros(size(EbN0));
BER_16QAM_coded_sameTx = zeros(size(EbN0));

% 16-QAM parameters
M_16QAM = 16;
k_16QAM = log2(M_16QAM); % 4 bits per symbol
Es_16QAM = k_16QAM * Eb; % Energy per symbol = 4*Eb

% Define 16-QAM constellation with Gray mapping
% Constellation points: ( $\pm 1 \pm j1$ ,  $\pm 1 \pm j3$ ,  $\pm 3 \pm j1$ ,  $\pm 3 \pm j3$ ) /  $\sqrt{10}$ 
% Gray mapping order
constellation_points = zeros(1, 16);
gray_mapping_16QAM = [0, 1, 3, 2, 4, 5, 7, 6, 12, 13, 15, 14, 8, 9, 11, 10]
+ 1;

% Create constellation in natural order
temp_const = zeros(1, 16);
idx = 1;
for q_val = [-3, -1, 1, 3]
for i_val = [-3, -1, 1, 3]
temp_const(idx) = (i_val + 1j*q_val) / sqrt(10);
idx = idx + 1;
end
end

% Apply Gray mapping
for i = 1:16
constellation_points(gray_mapping_16QAM(i)) = temp_const(i);
end

for snrIdx = 1:length(EbN0)
EbN0_current = EbN0(snrIdx);
No = Eb / EbN0_current;
sigma = sqrt(No/2);
% Generate random bits
b_k = randi([0 1], 1, numBits);
%% Uncoded 16-QAM
numSymbols = numBits / 4;
% Reshape bits into groups of 4
bits_reshaped = reshape(b_k, 4, numSymbols)';
% Modulate
symbols = zeros(numSymbols, 1);
for i = 1:numSymbols
bits = bits_reshaped(i, :);
dec_val = bits(1)*8 + bits(2)*4 + bits(3)*2 + bits(4);
gray_idx = gray_mapping_16QAM(dec_val + 1);
symbols(i) = constellation_points(gray_idx) * sqrt(Es_16QAM);
end
% Rayleigh fading channel
h = (randn(numSymbols, 1) + 1j*randn(numSymbols, 1)) * sqrt(1/2);
% AWGN
n = (randn(numSymbols, 1) + 1j*randn(numSymbols, 1)) * sigma;
% Received signal
y = h .* symbols + n;
% Channel compensation (perfect CSI)

```

```

symbols_hat = y ./ h;
% Demodulation (minimum distance)
b_hat = zeros(1, numBits);
for i = 1:numSymbols
    r = symbols_hat(i) / sqrt(Es_16QAM);
    distances = zeros(1, 16);
    for j = 1:16
        distances(j) = abs(r - constellation_points(j));
    end
    [~, min_idx] = min(distances);
    const_idx = find(gray_mapping_16QAM == min_idx) - 1;
    b_hat((i-1)*4+1:i*4) = [bitget(const_idx, 4), bitget(const_idx, 3), ...
        bitget(const_idx, 2), bitget(const_idx, 1)];
end
BER_16QAM_uncoded(snrIdx) = sum(b_hat ~= b_k) / numBits;
%% Coded 16-QAM (Rate 1/5 Repetition Code)
b_coded = repelem(b_k, 5);
numCodedBits = length(b_coded);
numCodedSymbols = numCodedBits / 4;
b_coded_reshaped = reshape(b_coded, 4, numCodedSymbols)';
symbols_coded = zeros(numCodedSymbols, 1);
for i = 1:numCodedSymbols
    bits = b_coded_reshaped(i, :);
    dec_val = bits(1)*8 + bits(2)*4 + bits(3)*2 + bits(4);
    gray_idx = gray_mapping_16QAM(dec_val + 1);
    symbols_coded(i) = constellation_points(gray_idx) * sqrt(Es_16QAM);
end
h_coded = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1)) *
    sqrt(1/2);
n_coded = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1)) *
    sigma;
y_coded = h_coded .* symbols_coded + n_coded;
symbols_coded_hat = y_coded ./ h_coded;
b_coded_hat = zeros(1, numCodedBits);
for i = 1:numCodedSymbols
    r = symbols_coded_hat(i) / sqrt(Es_16QAM);
    distances = zeros(1, 16);
    for j = 1:16
        distances(j) = abs(r - constellation_points(j));
    end
    [~, min_idx] = min(distances);
    const_idx = find(gray_mapping_16QAM == min_idx) - 1;
    b_coded_hat((i-1)*4+1:i*4) = [bitget(const_idx, 4), bitget(const_idx,
        3), ...
        bitget(const_idx, 2), bitget(const_idx, 1)];
end
b_decoded = zeros(1, numBits);
for i = 1:numBits
    segment = b_coded_hat((i-1)*5+1:i*5);
    b_decoded(i) = sum(segment) >= 3;
end
BER_16QAM_coded(snrIdx) = sum(b_decoded ~= b_k) / numBits;
%% Coded 16-QAM with Same Information Energy
symbols_coded_sameInfo = symbols_coded * sqrt(code_rate);
y_coded_sameInfo = h_coded .* symbols_coded_sameInfo + n_coded;
symbols_coded_hat_sameInfo = y_coded_sameInfo ./ h_coded;
b_coded_hat_sameInfo = zeros(1, numCodedBits);

```

```

for i = 1:numCodedSymbols
r = symbols_coded_hat_sameInfo(i) / sqrt(Es_16QAM * code_rate);
distances = zeros(1, 16);
for j = 1:16
distances(j) = abs(r - constellation_points(j));
end
[~, min_idx] = min(distances);
const_idx = find(gray_mapping_16QAM == min_idx) - 1;
b_coded_hat_sameInfo((i-1)*4+1:i*4) = [bitget(const_idx, 4),
bitget(const_idx, 3), ...
bitget(const_idx, 2), bitget(const_idx, 1)];
end
b_decoded_sameInfo = zeros(1, numBits);
for i = 1:numBits
segment = b_coded_hat_sameInfo((i-1)*5+1:i*5);
b_decoded_sameInfo(i) = sum(segment) >= 3;
end
BER_16QAM_coded_sameInfo(snrIdx) = sum(b_decoded_sameInfo ~= b_k) / numBits;
%% Coded 16-QAM with Same Transmission Energy
Eb_coded = Eb * code_rate;
No_coded = Eb_coded / EbN0_current;
sigma_coded = sqrt(No_coded/2);
symbols_coded_sameTx = symbols_coded * sqrt(code_rate);
n_coded_sameTx = (randn(numCodedSymbols, 1) + 1j*randn(numCodedSymbols, 1))
* sigma_coded;
y_coded_sameTx = h_coded .* symbols_coded_sameTx + n_coded_sameTx;
symbols_coded_hat_sameTx = y_coded_sameTx ./ h_coded;
b_coded_hat_sameTx = zeros(1, numCodedBits);
for i = 1:numCodedSymbols
r = symbols_coded_hat_sameTx(i) / sqrt(Es_16QAM * code_rate);
distances = zeros(1, 16);
for j = 1:16
distances(j) = abs(r - constellation_points(j));
end
[~, min_idx] = min(distances);
const_idx = find(gray_mapping_16QAM == min_idx) - 1;
b_coded_hat_sameTx((i-1)*4+1:i*4) = [bitget(const_idx, 4), bitget(const_idx,
3), ...
bitget(const_idx, 2), bitget(const_idx, 1)];
end
b_decoded_sameTx = zeros(1, numBits);
for i = 1:numBits
segment = b_coded_hat_sameTx((i-1)*5+1:i*5);
b_decoded_sameTx(i) = sum(segment) >= 3;
end
BER_16QAM_coded_sameTx(snrIdx) = sum(b_decoded_sameTx ~= b_k) / numBits;
end

%% Plot Results
fprintf('Plotting results...\n');
% Plot 1: Uncoded BPSK vs Coded BPSK for same info Energy
figure(1);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded BPSK');
hold on;
semilogy(EbN0_dB, BER_BPSK_coded_sameInfo, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded BPSK (Same Info Energy)');

```

```

grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('BPSK: Same Information Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 2: Uncoded BPSK vs Coded BPSK for same Transmission Energy
figure(2);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded BPSK');
hold on;
semilogy(EbN0_dB, BER_BPSK_coded_sameTx, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded BPSK (Same Tx Energy)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('BPSK: Same Transmission Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 3: Uncoded QPSK vs Coded QPSK for same info Energy
figure(3);
semilogy(EbN0_dB, BER_QPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded QPSK');
hold on;
semilogy(EbN0_dB, BER_QPSK_coded_sameInfo, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded QPSK (Same Info Energy)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('QPSK: Same Information Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 4: Uncoded QPSK vs Coded QPSK for same Transmission Energy
figure(4);
semilogy(EbN0_dB, BER_QPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded QPSK');
hold on;
semilogy(EbN0_dB, BER_QPSK_coded_sameTx, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded QPSK (Same Tx Energy)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('QPSK: Same Transmission Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 5: Uncoded 16-QAM vs Coded 16-QAM for same info Energy
figure(5);
semilogy(EbN0_dB, BER_16QAM_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded 16-QAM');
hold on;
semilogy(EbN0_dB, BER_16QAM_coded_sameInfo, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded 16-QAM (Same Info Energy)');

```

```

grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('16-QAM: Same Information Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 6: Uncoded 16-QAM vs Coded 16-QAM for same Transmission Energy
figure(6);
semilogy(EbN0_dB, BER_16QAM_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'Uncoded 16-QAM');
hold on;
semilogy(EbN0_dB, BER_16QAM_coded_sameTx, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded 16-QAM (Same Tx Energy)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('16-QAM: Same Transmission Energy Comparison');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 7: BPSK vs QPSK vs 16-QAM uncoded for same Transmission Energy
figure(7);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'BPSK');
hold on;
semilogy(EbN0_dB, BER_QPSK_uncoded, 'r-s', 'LineWidth', 2, 'DisplayName',
'QPSK');
semilogy(EbN0_dB, BER_16QAM_uncoded, 'g-^', 'LineWidth', 2, 'DisplayName',
'16-QAM');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Uncoded Modulation Comparison (Same Transmission Energy)');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 8: BPSK vs QPSK vs 16-QAM uncoded for same info Energy
figure(8);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 2, 'DisplayName',
'BPSK');
hold on;
semilogy(EbN0_dB, BER_QPSK_uncoded, 'r-s', 'LineWidth', 2, 'DisplayName',
'QPSK');
semilogy(EbN0_dB, BER_16QAM_uncoded, 'g-^', 'LineWidth', 2, 'DisplayName',
'16-QAM');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Uncoded Modulation Comparison (Same Information Energy)');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 9: BPSK vs QPSK vs 16-QAM coded for same Transmission Energy
figure(9);
semilogy(EbN0_dB, BER_BPSK_coded_sameTx, 'b-o', 'LineWidth', 2,
'DisplayName', 'Coded BPSK');

```

```

hold on;
semilogy(EbN0_dB, BER_QPSK_coded_sameTx, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded QPSK');
semilogy(EbN0_dB, BER_16QAM_coded_sameTx, 'g-^', 'LineWidth', 2,
'DisplayName', 'Coded 16-QAM');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Coded Modulation Comparison (Same Transmission Energy)');
legend('Location', 'best');
ylim([1e-6 1]);

% Plot 10: BPSK vs QPSK vs 16-QAM coded for same info Energy
figure(10);
semilogy(EbN0_dB, BER_BPSK_coded_sameInfo, 'b-o', 'LineWidth', 2,
'DisplayName', 'Coded BPSK');
hold on;
semilogy(EbN0_dB, BER_QPSK_coded_sameInfo, 'r-s', 'LineWidth', 2,
'DisplayName', 'Coded QPSK');
semilogy(EbN0_dB, BER_16QAM_coded_sameInfo, 'g-^', 'LineWidth', 2,
'DisplayName', 'Coded 16-QAM');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Coded Modulation Comparison (Same Information Energy)');
legend('Location', 'best');
ylim([1e-6 1]);
%% Additional Comprehensive Comparison Plots

% Plot 11: All schemes (coded and uncoded) for same information energy
figure(11);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded BPSK');
hold on;
semilogy(EbN0_dB, BER_BPSK_coded_sameInfo, 'b--s', 'LineWidth', 1.5,
'DisplayName', 'Coded BPSK (Same Info)');
semilogy(EbN0_dB, BER_QPSK_uncoded, 'r-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded QPSK');
semilogy(EbN0_dB, BER_QPSK_coded_sameInfo, 'r--s', 'LineWidth', 1.5,
'DisplayName', 'Coded QPSK (Same Info)');
semilogy(EbN0_dB, BER_16QAM_uncoded, 'g-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded 16-QAM');
semilogy(EbN0_dB, BER_16QAM_coded_sameInfo, 'g--s', 'LineWidth', 1.5,
'DisplayName', 'Coded 16-QAM (Same Info)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Comprehensive Comparison: All Schemes with Same Information Energy');
legend('Location', 'best', 'NumColumns', 2);
ylim([1e-6 1]);
xlim([0 20]);

% Plot 12: All schemes (coded and uncoded) for same transmission energy
figure(12);
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded BPSK');
hold on;

```

```

semilogy(EbN0_dB, BER_BPSK_coded_sameTx, 'b--s', 'LineWidth', 1.5,
'DisplayName', 'Coded BPSK (Same Tx)');
semilogy(EbN0_dB, BER_QPSK_uncoded, 'r-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded QPSK');
semilogy(EbN0_dB, BER_QPSK_coded_sameTx, 'r--s', 'LineWidth', 1.5,
'DisplayName', 'Coded QPSK (Same Tx)');
semilogy(EbN0_dB, BER_16QAM_uncoded, 'g-o', 'LineWidth', 1.5, 'DisplayName',
'Uncoded 16-QAM');
semilogy(EbN0_dB, BER_16QAM_coded_sameTx, 'g--s', 'LineWidth', 1.5,
'DisplayName', 'Coded 16-QAM (Same Tx)');
grid on;
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Comprehensive Comparison: All Schemes with Same Transmission
Energy');
legend('Location', 'best', 'NumColumns', 2);
ylim([1e-6 1]);
xlim([0 20]);

%% Figure 13: ALL 12 Lines on Same Graph (Combined Comparison)
figure(13);
set(gcf, 'Position', [100, 100, 1000, 800]);

% Plot all 12 lines on the same graph
semilogy(EbN0_dB, BER_BPSK_uncoded, 'b-o', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Uncoded BPSK');
hold on;
semilogy(EbN0_dB, BER_BPSK_coded_sameInfo, 'b--s', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded BPSK (Same Info Energy)');
semilogy(EbN0_dB, BER_BPSK_coded_sameTx, 'b-.^', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded BPSK (Same Tx Energy)');

semilogy(EbN0_dB, BER_QPSK_uncoded, 'r-o', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Uncoded QPSK');
semilogy(EbN0_dB, BER_QPSK_coded_sameInfo, 'r--s', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded QPSK (Same Info Energy)');
semilogy(EbN0_dB, BER_QPSK_coded_sameTx, 'r-.^', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded QPSK (Same Tx Energy)');

semilogy(EbN0_dB, BER_16QAM_uncoded, 'g-o', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Uncoded 16-QAM');
semilogy(EbN0_dB, BER_16QAM_coded_sameInfo, 'g--s', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded 16-QAM (Same Info Energy)');
semilogy(EbN0_dB, BER_16QAM_coded_sameTx, 'g-.^', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded 16-QAM (Same Tx Energy)');

grid on;
xlabel('Eb/No (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('Complete BER Comparison: All Modulation Schemes with Rate 1/5
Repetition Code', ...
'FontSize', 14, 'FontWeight', 'bold');

% Create a comprehensive legend

```

```

legend('Location', 'best', 'FontSize', 9, 'NumColumns', 2);

% Set axis limits
ylim([1e-6 1]);
xlim([0 20]);

% Add grid styling
set(gca, 'FontSize', 11, 'GridLineStyle', '--', 'GridAlpha', 0.3);

%% Figure 14: BPSK - 4 Energy Scenarios (2 Uncoded + 2 Coded)
figure(14);
set(gcf, 'Position', [100, 100, 900, 700]);

% Plot BPSK scenarios
% For uncoded, same energy per info bit = same energy per transmission bit
(no coding)
semilogy(EbN0_dB, BER_BPSK_uncoded, 'k-o', 'LineWidth', 2.5, 'MarkerSize',
10, 'DisplayName', 'Uncoded (Same Energy per Info Bit)');
hold on;
% Plot the same uncoded curve with different style for visual distinction
semilogy(EbN0_dB, BER_BPSK_uncoded, 'k--s', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Uncoded (Same Energy per Tx Bit)');

% Coded scenarios
semilogy(EbN0_dB, BER_BPSK_coded_sameInfo, 'b-.^', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded (Same Energy per Info Bit)');
semilogy(EbN0_dB, BER_BPSK_coded_sameTx, 'r:*', 'LineWidth', 2,
'MarkerSize', 10, 'DisplayName', 'Coded (Same Energy per Tx Bit)');

grid on;
xlabel('Eb/No (dB)', 'FontSize', 13, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 13, 'FontWeight', 'bold');
title('BPSK: Energy per Information Bit vs Transmission Bit', 'FontSize',
14, 'FontWeight', 'bold');

legend('Location', 'southwest', 'FontSize', 11);
ylim([1e-6 1]);
xlim([0 20]);
set(gca, 'FontSize', 12, 'GridLineStyle', '--', 'GridAlpha', 0.3);

%% Figure 15: QPSK - 4 Energy Scenarios (2 Uncoded + 2 Coded)
figure(15);
set(gcf, 'Position', [200, 100, 900, 700]);

% Plot QPSK scenarios
semilogy(EbN0_dB, BER_QPSK_uncoded, 'k-o', 'LineWidth', 2.5, 'MarkerSize',
10, 'DisplayName', 'Uncoded (Same Energy per Info Bit)');
hold on;
semilogy(EbN0_dB, BER_QPSK_uncoded, 'k--s', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Uncoded (Same Energy per Tx Bit)');

semilogy(EbN0_dB, BER_QPSK_coded_sameInfo, 'b-.^', 'LineWidth', 2,

```

```

'MarkerSize', 8, 'DisplayName', 'Coded (Same Energy per Info Bit)');
semilogy(EbN0_dB, BER_QPSK_coded_sameTx, 'r:*', 'LineWidth', 2,
'MarkerSize', 10, 'DisplayName', 'Coded (Same Energy per Tx Bit)');

grid on;
xlabel('Eb/No (dB)', 'FontSize', 13, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 13, 'FontWeight', 'bold');
title('QPSK: Energy per Information Bit vs Transmission Bit', 'FontSize',
14, 'FontWeight', 'bold');

legend('Location', 'southwest', 'FontSize', 11);
ylim([1e-6 1]);
xlim([0 20]);
set(gca, 'FontSize', 12, 'GridLineStyle', '--', 'GridAlpha', 0.3);

%% Figure 16: 16-QAM - 4 Energy Scenarios (2 Uncoded + 2 Coded)
figure(16);
set(gcf, 'Position', [300, 100, 900, 700]);

% Plot 16-QAM scenarios
semilogy(EbN0_dB, BER_16QAM_uncoded, 'k-o', 'LineWidth', 2.5, 'MarkerSize',
10, 'DisplayName', 'Uncoded (Same Energy per Info Bit)');
hold on;
semilogy(EbN0_dB, BER_16QAM_uncoded, 'k--s', 'LineWidth', 2, 'MarkerSize',
8, 'DisplayName', 'Uncoded (Same Energy per Tx Bit)');

semilogy(EbN0_dB, BER_16QAM_coded_sameInfo, 'b-.^', 'LineWidth', 2,
'MarkerSize', 8, 'DisplayName', 'Coded (Same Energy per Info Bit)');
semilogy(EbN0_dB, BER_16QAM_coded_sameTx, 'r:*', 'LineWidth', 2,
'MarkerSize', 10, 'DisplayName', 'Coded (Same Energy per Tx Bit)');

grid on;
xlabel('Eb/No (dB)', 'FontSize', 13, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 13, 'FontWeight', 'bold');
title('16-QAM: Energy per Information Bit vs Transmission Bit', 'FontSize',
14, 'FontWeight', 'bold');

legend('Location', 'southwest', 'FontSize', 11);
ylim([1e-6 1]);
xlim([0 20]);
set(gca, 'FontSize', 12, 'GridLineStyle', '--', 'GridAlpha', 0.3);

end
%% ===== Problem 3
code=====%%
function problem_3 ()
clc; clear; close all;
%% ===== USER SELECTIONS =====
% Choose modulation scheme
modulation_choice = menu('Select Modulation Scheme:', 'QPSK', '16QAM');
if modulation_choice == 1
modulation = 'QPSK';
else
modulation = '16QAM';
end

```

```

% Choose channel type
channel_choice = menu('Select Channel Type:', 'Rayleigh Flat Fading',
'Frequency Selective');
if channel_choice == 1
channel_type = 'Rayleigh';
else
channel_type = 'Frequency Selective';
end
fprintf('\n===== Selected Parameters =====\n');
fprintf('Modulation: %s\n', modulation);
fprintf('Channel Type: %s\n', channel_type);
fprintf('===== \n\n');
%% ===== SIMULATION PARAMETERS =====
Nfft = 256; % IFFT/FFT
numOfdmSymbols = 1000; % OFDM
CP = 32; % Cyclic Prefix
Eb = 1;
% Channel parameters for Frequency Selective
numTaps = 4;
maxDelay = 10;
% Calculate bits per symbol
if strcmp(modulation, 'QPSK')
bitsPerSymbol = 2;
else
bitsPerSymbol = 4; % 16QAM
end
% Calculate total modulation symbols needed
total_mod_symbols = Nfft * numOfdmSymbols;
% SNR range
SNR_dB = -4:2:16;
%% ===== SIMULATE BOTH CODING SCHEMES =====
BER_no_coding = zeros(1, length(SNR_dB));
BER_repetition = zeros(1, length(SNR_dB));
% Loop over both coding schemes
coding_schemes = {'no_coding', 'repetition_1_5'};
for scheme_idx = 1:2
coding_scheme = coding_schemes{scheme_idx};
fprintf('\nSimulating: %s with %s modulation...\n', coding_scheme,
modulation);
if strcmp(coding_scheme, 'repetition_1_5')
repetition_rate = 5;
N_info = floor((total_mod_symbols * bitsPerSymbol) / repetition_rate);
N = N_info * repetition_rate;
else
repetition_rate = 1;
N = total_mod_symbols * bitsPerSymbol;
N_info = N;
end
% Ensure N is divisible by 256 for interleaver
N = floor(N/256) * 256;
if strcmp(coding_scheme, 'repetition_1_5')
N_info = floor(N / repetition_rate);
end
fprintf(' Information bits: %d\n', N_info);
fprintf(' Transmitted bits: %d\n', N);
%% Generate random bits
if strcmp(coding_scheme, 'repetition_1_5')

```

```

info_bits = randi([0 1], 1, N_info);
bits = zeros(1, N);
for i = 1:N_info
start_idx = (i-1)*repetition_rate + 1;
end_idx = i*repetition_rate;
bits(start_idx:end_idx) = repmat(info_bits(i), 1, repetition_rate);
end
else
info_bits = randi([0 1], 1, N);
bits = info_bits;
end
%% Interleaver (16x16)
num_interleaver_blocks = N / 256;
data = reshape(bits, 16, 16, num_interleaver_blocks);
data = permute(data, [2 1 3]);
data = reshape(data, 1, N);
%% Modulation Mapping
if strcmp(modulation, 'QPSK')
modulated_symbols = zeros(1, N/2);
m = 1;
for i = 1:2:N
if data(i)==1 && data(i+1)==1
modulated_symbols(m) = 1 + 1j;
elseif data(i)==0 && data(i+1)==1
modulated_symbols(m) = -1 + 1j;
elseif data(i)==1 && data(i+1)==0
modulated_symbols(m) = 1 - 1j;
elseif data(i)==0 && data(i+1)==0
modulated_symbols(m) = -1 - 1j;
end
m = m + 1;
end
else % 16QAM
modulated_symbols = zeros(1, N/4);
m = 1;
for i = 1:4:N
b0 = data(i); b1 = data(i+1); b2 = data(i+2); b3 = data(i+3);
if b0==0 && b1==0
I = -3;
elseif b0==0 && b1==1
I = -1;
elseif b0==1 && b1==1
I = 1;
else
I = 3;
end
if b2==0 && b3==0
Q = -3;
elseif b2==0 && b3==1
Q = -1;
elseif b2==1 && b3==1
Q = 1;
else
Q = 3;
end
modulated_symbols(m) = I + 1j*Q;
m = m + 1;
end

```

```

end
modulated_symbols = modulated_symbols / sqrt(10);
end
% Pad or truncate to fit OFDM symbols
if length(modulated_symbols) > total_mod_symbols
modulated_symbols = modulated_symbols(1:total_mod_symbols);
elseif length(modulated_symbols) < total_mod_symbols
padding_needed = total_mod_symbols - length(modulated_symbols);
modulated_symbols = [modulated_symbols, zeros(1, padding_needed)];
end
%% IFFT + Cyclic Prefix
ifft_sig = reshape(modulated_symbols, Nfft, numOfdmSymbols);
ifft_sig = ifft(ifft_sig, Nfft, 1);
ofdm_symbols = zeros((Nfft+CP)*numOfdmSymbols, 1);
for k = 1:numOfdmSymbols
ofdm_symbols((k-1)*(Nfft+CP)+1 : k*(Nfft+CP)) = ...
[ifft_sig(end-CP+1:end, k); ifft_sig(:, k)];
end
%% Loop over SNR values
for z = 1:length(SNR_dB)
SNR = 10^(SNR_dB(z)/10);
No = Eb/SNR;
%% Channel Model
if strcmp(channel_type, 'Rayleigh')
% ===== RAYLEIGH FLAT FADING =====
% Noise
n_deviation = sqrt(No/2);
noise = n_deviation*(randn(length(ofdm_symbols), 1) +
1j*randn(length(ofdm_symbols), 1));
% Rayleigh channel - ?????? ??? ?? ?????? ??????
h_deviation = sqrt(0.5);
channel = h_deviation*(randn(length(ofdm_symbols), 1) +
1j*randn(length(ofdm_symbols), 1));
% Received signal
y_rec = channel.*ofdm_symbols + noise;
% Equalization (Zero Forcing)
y = y_rec./channel;
% Remove cyclic prefix
y_decyclic = zeros(Nfft, numOfdmSymbols);
for k = 1:numOfdmSymbols
y_decyclic(:, k) = y((k-1)*(Nfft+CP)+CP+1 : k*(Nfft+CP));
end
else
% ===== FREQUENCY SELECTIVE =====
% Generate channel taps
delays = randi([0, maxDelay], 1, numTaps);
delays = sort(unique(delays));
numTaps_current = length(delays);
channel_taps = (randn(1, numTaps_current) + 1j*randn(1, numTaps_current)) /
sqrt(2);
pdp = exp(-delays/maxDelay);
channel_taps = channel_taps .* sqrt(pdp);
channel_taps = channel_taps / sqrt(sum(abs(channel_taps).^2));
% Create channel impulse response
channel_ir = zeros(maxDelay+1, 1);
channel_ir(delays+1) = channel_taps;
% Convolve with channel

```

```

padded_signal = [ofdm_symbols; zeros(maxDelay, 1)];
conv_signal = conv(padded_signal, channel_ir);
y_rec_time = conv_signal(1:length(ofdm_symbols));
% Add AWGN Noise
n_deviation = sqrt(No/2);
noise = n_deviation*(randn(length(y_rec_time), 1) +
1j*randn(length(y_rec_time), 1));
y_rec = y_rec_time + noise;
% Remove cyclic prefix
y_deccyclic = zeros(Nfft, numOfdmSymbols);
for k = 1:numOfdmSymbols
start_idx = (k-1)*(Nfft+CP) + CP + 1;
end_idx = k*(Nfft+CP);
y_deccyclic(:, k) = y_rec(start_idx:end_idx);
end
% FFT and Equalization for frequency selective
fft_sig = fft(y_deccyclic, Nfft, 1);
% Frequency selective - use known channel frequency response
channel_freq = fft(channel_ir, Nfft);
fft_sig_eq = zeros(size(fft_sig));
for k = 1:numOfdmSymbols
fft_sig_eq(:, k) = fft_sig(:, k) ./ channel_freq;
end
% Skip the rest of processing for frequency selective and continue
fft_sig_eq = reshape(fft_sig_eq, 1, []);
% Take only the actual transmitted symbols
actual_symbols = length(modulated_symbols);
if length(fft_sig_eq) > actual_symbols
fft_sig_eq = fft_sig_eq(1:actual_symbols);
end
end
%% FFT for Rayleigh channel
if strcmp(channel_type, 'Rayleigh')
fft_sig = fft(y_deccyclic, Nfft, 1);
fft_sig_eq = reshape(fft_sig, 1, []);
% Take only the actual transmitted symbols
actual_symbols = length(modulated_symbols);
if length(fft_sig_eq) > actual_symbols
fft_sig_eq = fft_sig_eq(1:actual_symbols);
end
end
%% Demapping
data_estimated = zeros(1, N);
if strcmp(modulation, 'QPSK')
s = 1;
num_symbols_to_demap = min(length(fft_sig_eq), N/2);
for i = 1:num_symbols_to_demap
if real(fft_sig_eq(i))>=0 && imag(fft_sig_eq(i))>=0
data_estimated(s) = 1; data_estimated(s+1) = 1;
elseif real(fft_sig_eq(i))<0 && imag(fft_sig_eq(i))>=0
data_estimated(s) = 0; data_estimated(s+1) = 1;
elseif real(fft_sig_eq(i))<0 && imag(fft_sig_eq(i))<0
data_estimated(s) = 0; data_estimated(s+1) = 0;
else
data_estimated(s) = 1; data_estimated(s+1) = 0;
end
s = s + 2;

```

```

end
else % 16QAM
if strcmp(modulation, '16QAM')
fft_sig_eq = fft_sig_eq * sqrt(10);
end
s = 1;
num_symbols_to_demap = min(length(fft_sig_eq), N/4);
for i = 1:num_symbols_to_demap
I = real(fft_sig_eq(i));
Q = imag(fft_sig_eq(i));
% Demap in-phase component
if I >= 2
data_estimated(s) = 1; data_estimated(s+1) = 0;
elseif I >= 0
data_estimated(s) = 1; data_estimated(s+1) = 1;
elseif I >= -2
data_estimated(s) = 0; data_estimated(s+1) = 1;
else
data_estimated(s) = 0; data_estimated(s+1) = 0;
end
% Demap quadrature component
if Q >= 2
data_estimated(s+2) = 1; data_estimated(s+3) = 0;
elseif Q >= 0
data_estimated(s+2) = 1; data_estimated(s+3) = 1;
elseif Q >= -2
data_estimated(s+2) = 0; data_estimated(s+3) = 1;
else
data_estimated(s+2) = 0; data_estimated(s+3) = 0;
end
s = s + 4;
end
end
%% Deinterleaving
deInterleaver_data = reshape(data_estimated, 16, 16,
num_interleaver_blocks);
deInterleaver_data = permute(deInterleaver_data, [2 1 3]);
deInterleaver_data = reshape(deInterleaver_data, 1, []);
%% Decoding (for repetition coding)
if strcmp(coding_scheme, 'repetition_1_5')
decoded_bits = zeros(1, N_info);
for i = 1:N_info
start_idx = (i-1)*repetition_rate + 1;
end_idx = i*repetition_rate;
repeated_bits = deInterleaver_data(start_idx:end_idx);
num_ones = sum(repeated_bits);
if num_ones > repetition_rate/2
decoded_bits(i) = 1;
elseif num_ones < repetition_rate/2
decoded_bits(i) = 0;
else
decoded_bits(i) = randi([0 1]);
end
end
% Calculate BER
error = sum(decoded_bits ~= info_bits(1:N_info));
if scheme_idx == 1

```

```

BER_no_coding(z) = error / N_info;
else
BER_repetition(z) = error / N_info;
end
else
error = sum(deInterleaver_data(1:N) ~= bits(1:N));
BER_no_coding(z) = error / N;
end
end
fprintf(' Simulation completed.\n');
end
%% ===== PLOT RESULTS =====
figure('Position', [100, 100, 900, 600]);
% Plot BER curves
semilogy(SNR_dB, BER_no_coding, '-ro', 'LineWidth', 2, 'MarkerSize', 8,
'MarkerFaceColor', 'r');
hold on;
semilogy(SNR_dB, BER_repetition, '-bs', 'LineWidth', 2, 'MarkerSize', 8,
'MarkerFaceColor', 'b');
xlabel('SNR (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
% Create title based on selections
title_str = sprintf('BER Performance: %s Modulation over %s Channel',
modulation, channel_type);
title(title_str, 'FontSize', 14, 'FontWeight', 'bold');
grid on;
legend('No Coding', 'Repetition Coding (1/5)', 'Location', 'Best');
set(gca, 'FontSize', 11);
% Add text box with simulation parameters
annotation_text = sprintf('Simulation Parameters:\nNFFT = %d\nOFDM Symbols
= %d\nCP Length = %d', ...
Nfft, numOfdmSymbols, CP);
annotation('textbox', [0.15, 0.6, 0.2, 0.15], 'String', annotation_text, ...
'FontSize', 10, 'BackgroundColor', 'white', 'EdgeColor', 'black');
%% ===== DISPLAY RESULTS TABLE =====
fprintf('\n===== BER Results =====\n');
fprintf('SNR (dB)\tNo Coding\tRepetition (1/5)\tImprovement\n');
fprintf('-----\n');
for i = 1:length(SNR_dB)
if BER_repetition(i) > 0
improvement = BER_no_coding(i)/BER_repetition(i);
fprintf('%d\t\t%.2e\t%.2e\t\t%.2f\n', ...
SNR_dB(i), BER_no_coding(i), BER_repetition(i), improvement);
else
fprintf('%d\t\t%.2e\t%.2e\t\t-\n', ...
SNR_dB(i), BER_no_coding(i), BER_repetition(i));
end
end
fprintf('===== \n');
end

```